UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

SISTEMAS DE BASES DE DATOS 2 Sección B

MANUAL DE EVIDENCIA DE DECISIONES TÉCNICAS Y ARQUITECTURA DE DATOS

Álvaro Gabriel Ceballos Gil - 202300673

Ottoniel Fabricio Vásquez Pineda 202307820

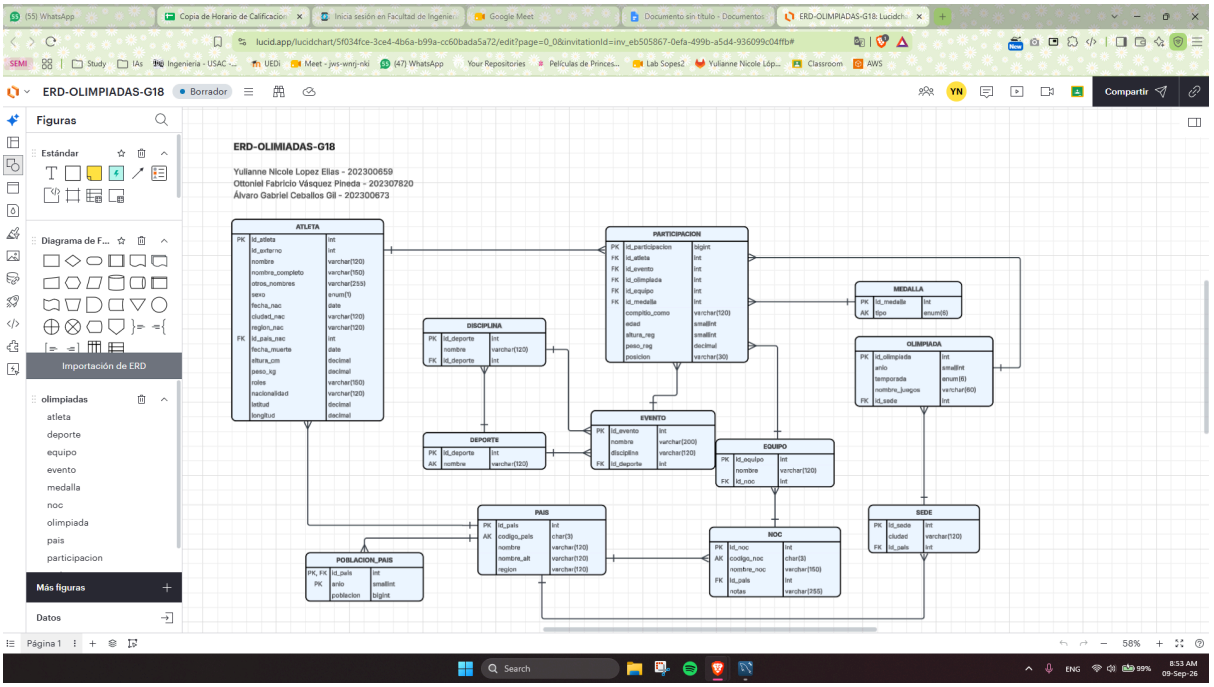
Yulianne Nicole López Elias - 202300659

GUATEMALA, 18 de septiembre de 2026

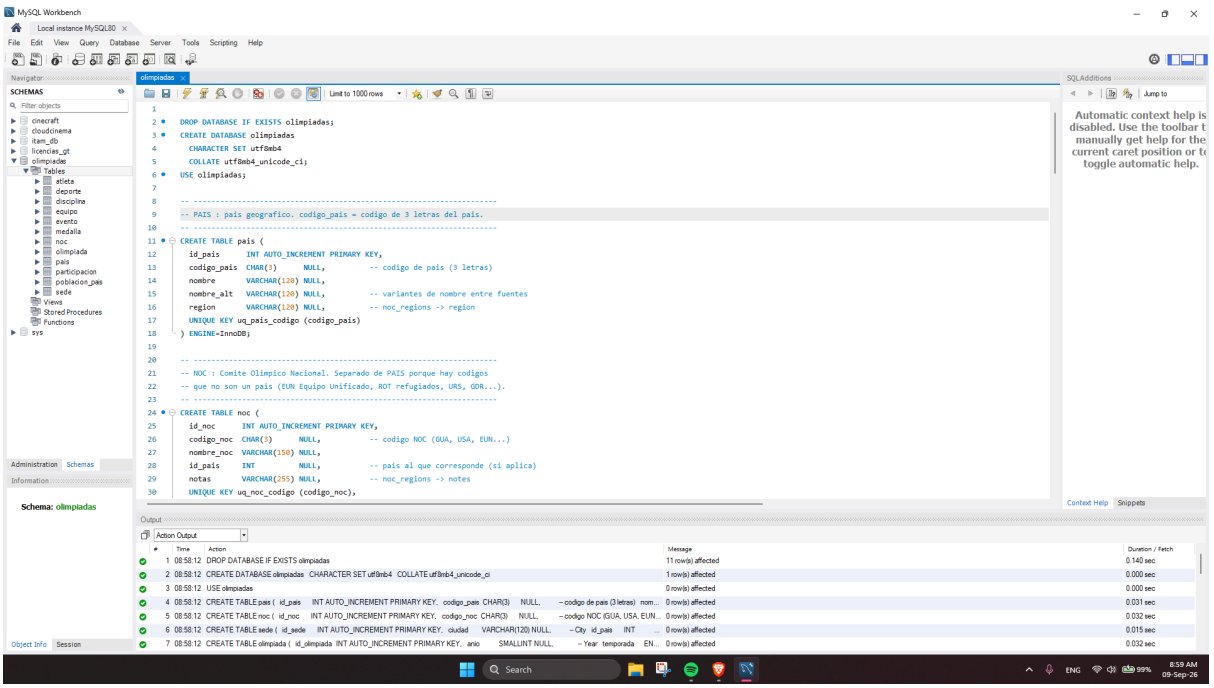
PROYECTO DE INTEGRACIÓN Y LIMPIEZA DE JUEGOS OLÍMPICOS

SISTEMAS DE BASES DE DATOS 2

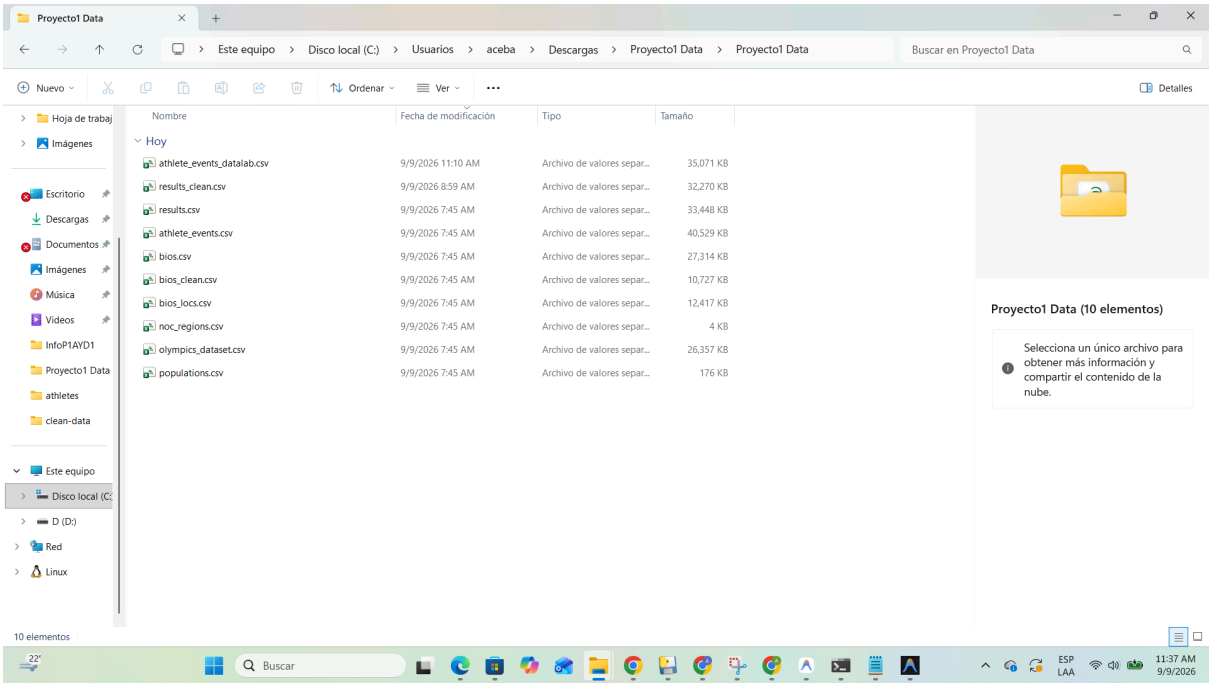
Creación ERD



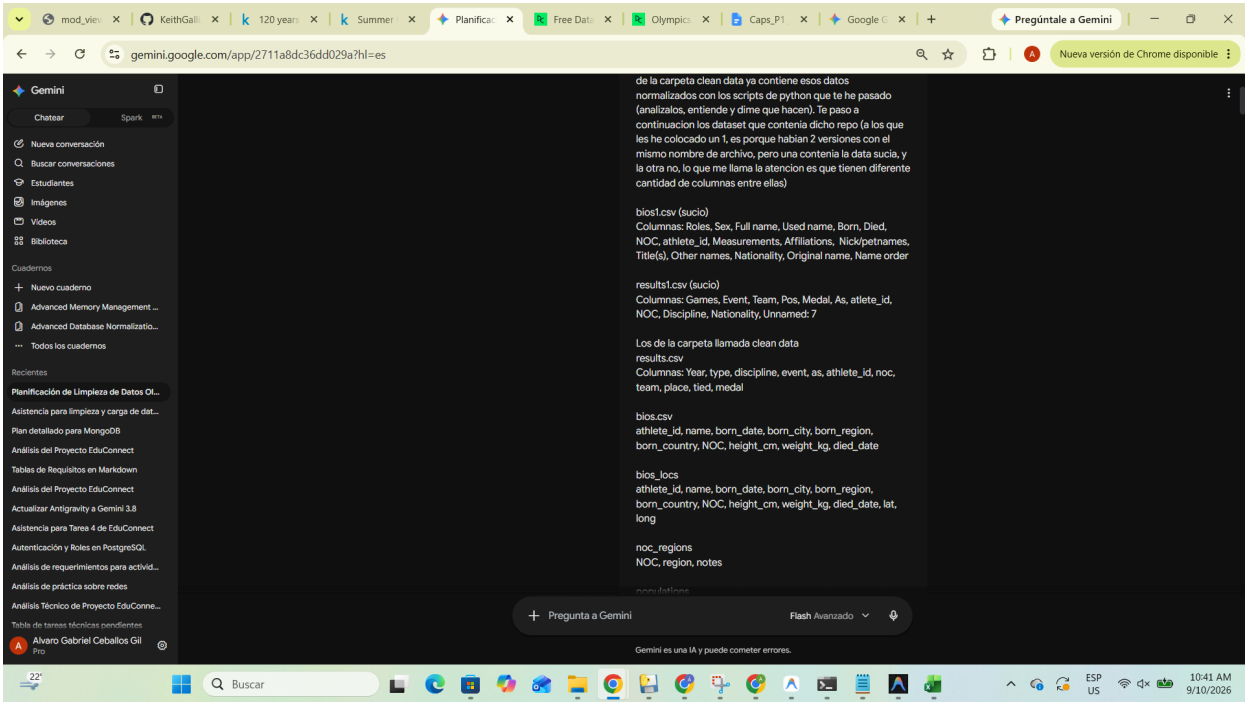
Creación de tablas



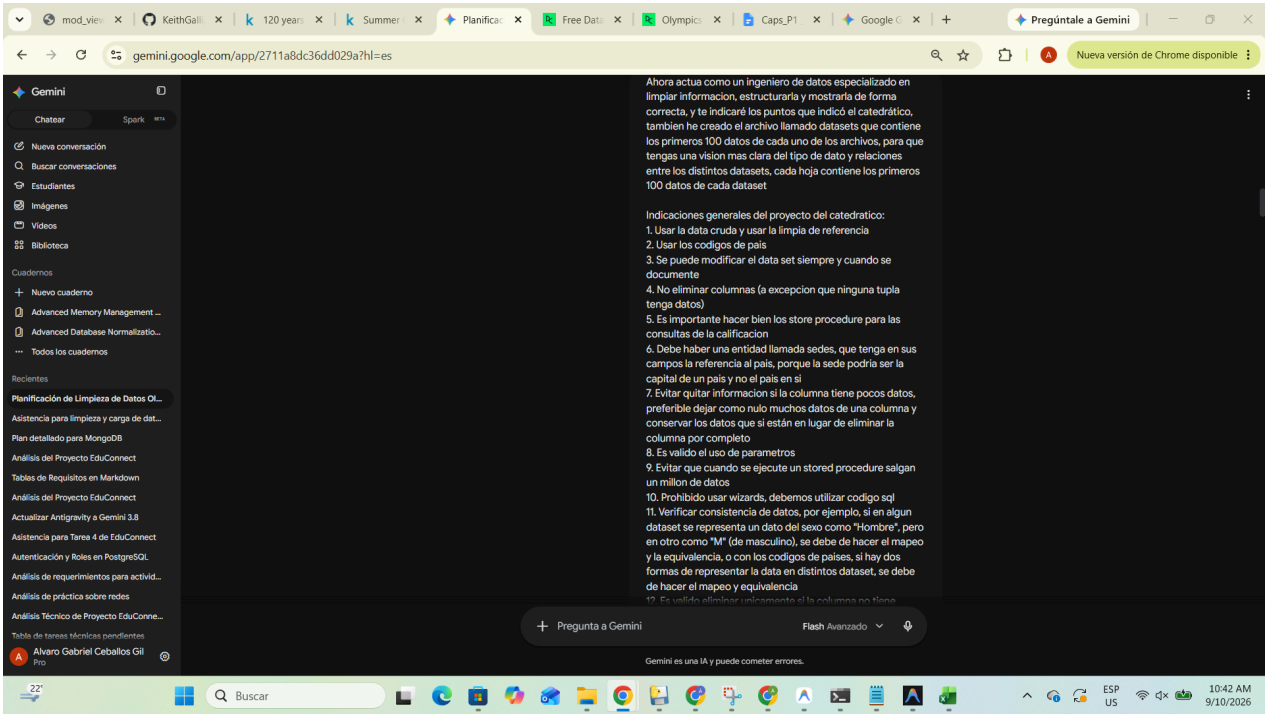
Recolección de data

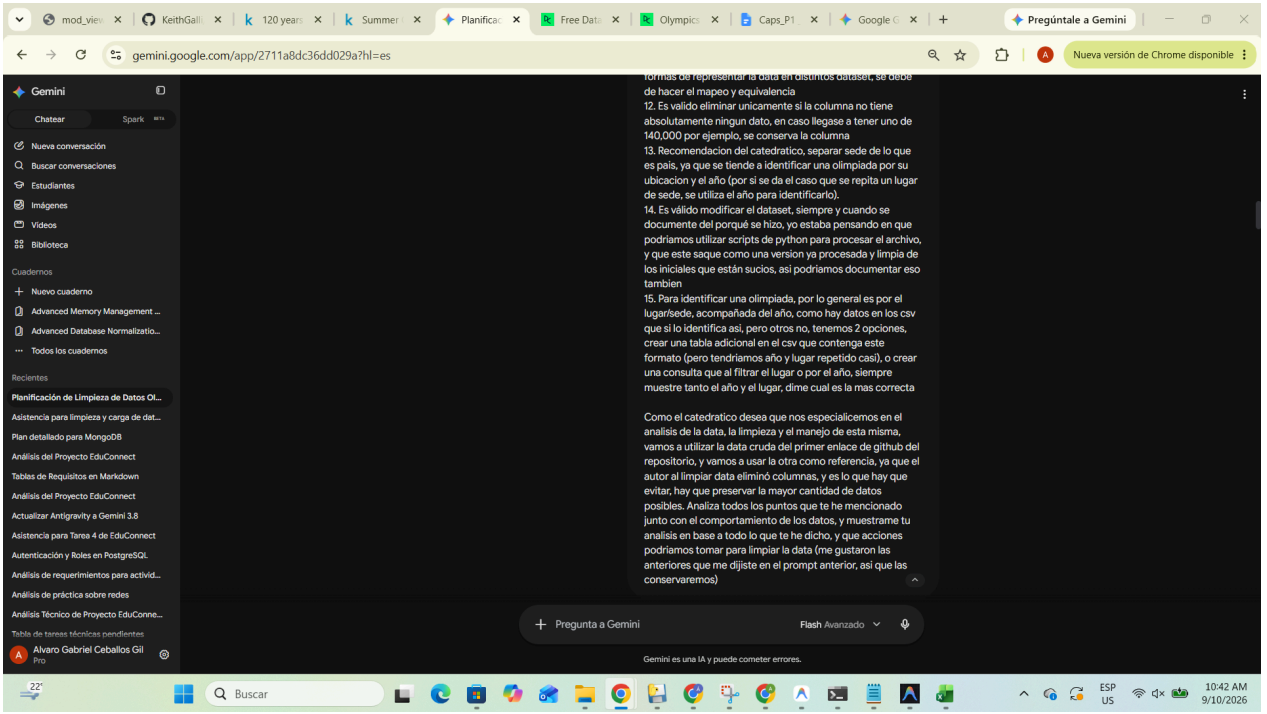


Prompts

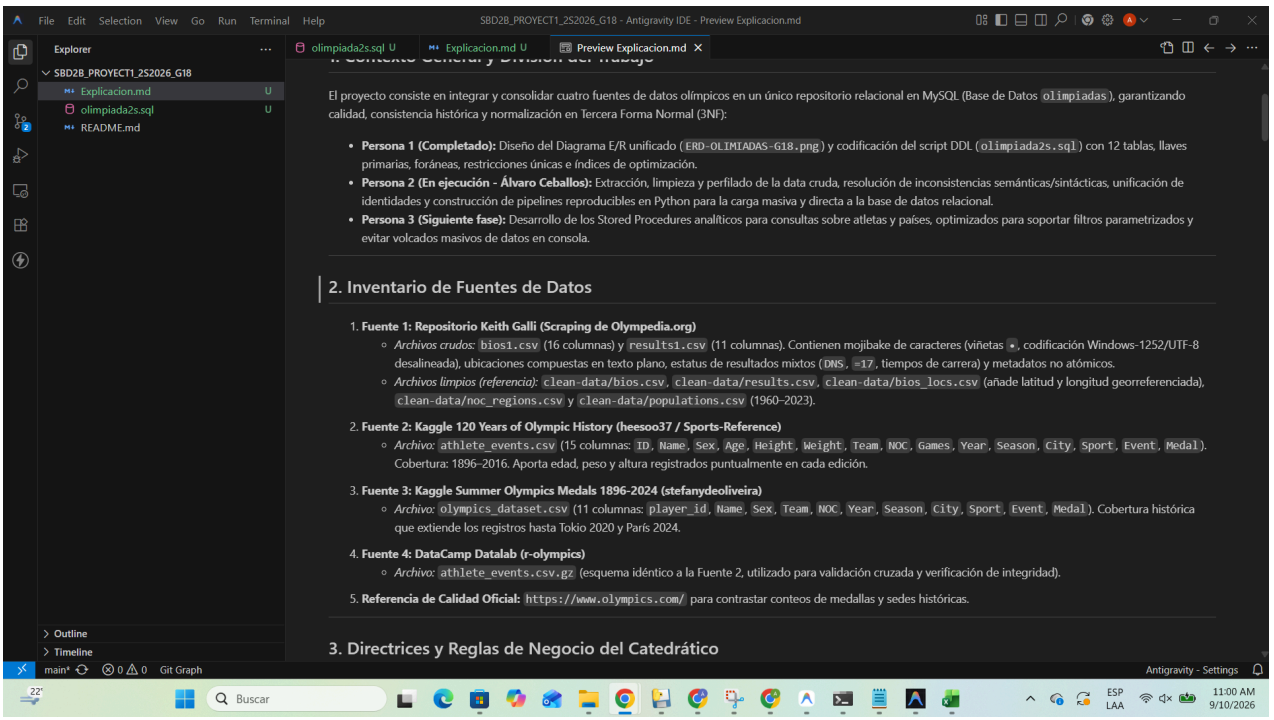


Prompt inicial con la explicación de la estructura del proyecto, indicando que data contiene cada uno de los csv, y qué decisiones se tomarán para la limpieza de los datos





Prompts donde listo todas las consideraciones técnicas y que pautas se tomarán en cuenta para el manejo de los distintos dataset



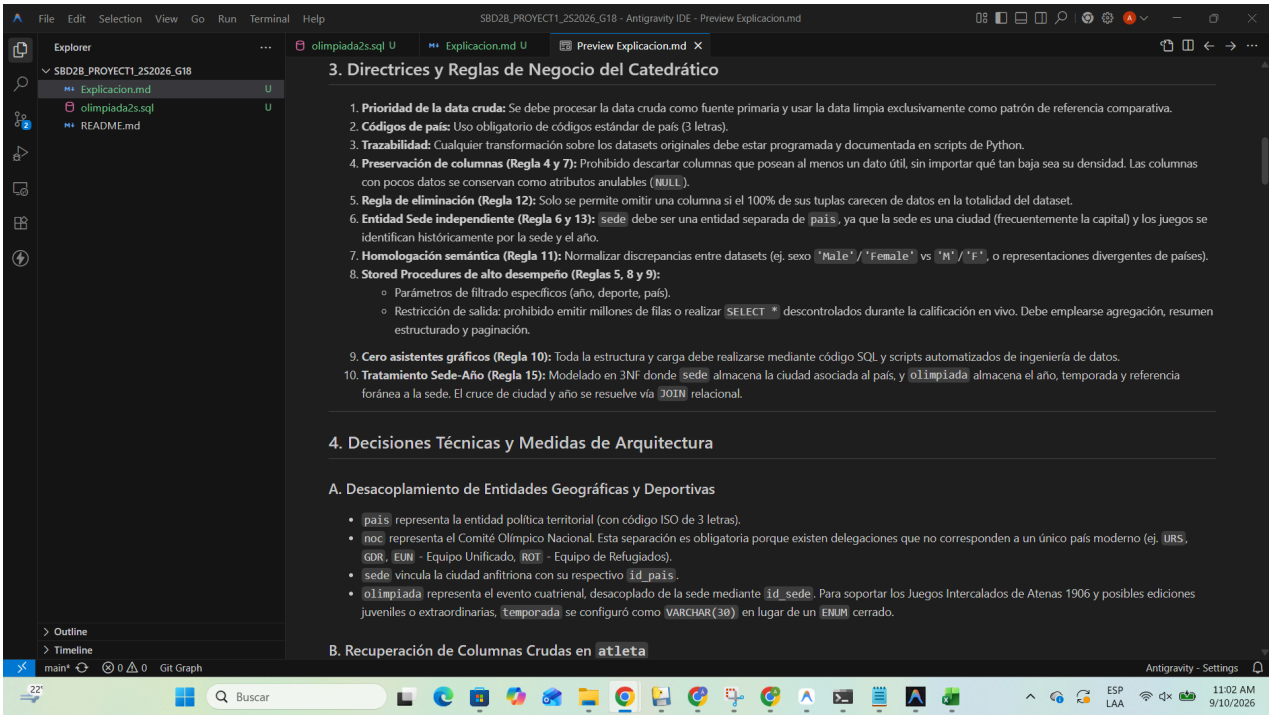
El proyecto consiste en integrar y consolidar cuatro fuentes de datos olímpicos en un único repositorio relacional en MySQL (Base de Datos `olimpiadas`), garantizando calidad, consistencia histórica y normalización en Tercera Forma Normal (3NF):

- **Persona 1 (Completado):** Diseño del Diagrama E/R unificado (`ERD-OLIMIADAS-G18.png`) y codificación del script DDL (`olimpiada2s.sql`) con 12 tablas, llaves primarias, foráneas, restricciones únicas e índices de optimización.
- **Persona 2 (En ejecución - Álvaro Ceballos):** Extracción, limpieza y perfilado de la data cruda, resolución de inconsistencias semánticas/sintácticas, unificación de identidades y construcción de pipelines reproducibles en Python para la carga masiva y directa a la base de datos relacional.
- **Persona 3 (Siguiente fase):** Desarrollo de los Stored Procedures analíticos para consultas sobre atletas y países, optimizados para soportar filtros parametrizados y evitar volcados masivos de datos en consola.

2. Inventario de Fuentes de Datos

1. **Fuente 1: Repositorio Keith Galli (Scraping de Olympedia.org)**
   - *Archivos crudos:* `bios1.csv` (16 columnas) y `results1.csv` (11 columnas). Contienen mojibake de caracteres (viñetas `•`, codificación Windows-1252/UTF-8 desalineada), ubicaciones compuestas en texto plano, estatus de resultados mixtos (`DNS`, `=17`, tiempos de carrera) y metadatos no atómicos.
   - *Archivos limpios (referencia):* `clean-data/bios.csv`, `clean-data/results.csv`, `clean-data/bios_locs.csv` (añade latitud y longitud georreferenciada), `clean-data/noc_regions.csv` y `clean-data/populations.csv` (1960–2023).
2. **Fuente 2: Kaggle 120 Years of Olympic History (heesoo37 / Sports-Reference)**
   - *Archivo:* `athlete_events.csv` (15 columnas: `ID`, `Name`, `Sex`, `Age`, `Height`, `Weight`, `Team`, `NOC`, `Games`, `Year`, `Season`, `City`, `Sport`, `Event`, `Medal`). Cobertura: 1896–2016. Aporta edad, peso y altura registrados puntualmente en cada edición.
3. **Fuente 3: Kaggle Summer Olympics Medals 1896-2024 (stefanydeoliveira)**
   - *Archivo:* `olympics_dataset.csv` (11 columnas: `player_id`, `Name`, `Sex`, `Team`, `NOC`, `Year`, `Season`, `City`, `Sport`, `Event`, `Medal`). Cobertura histórica que extiende los registros hasta Tokio 2020 y París 2024.
4. **Fuente 4: DataCamp Datalab (r-olympics)**
   - *Archivo:* `athlete_events.csv.gz` (esquema idéntico a la Fuente 2, utilizado para validación cruzada y verificación de integridad).
5. **Referencia de Calidad Oficial:** `https://www.olympics.com/` para contrastar conteos de medallas y sedes históricas.

3. Directrices y Reglas de Negocio del Catedrático



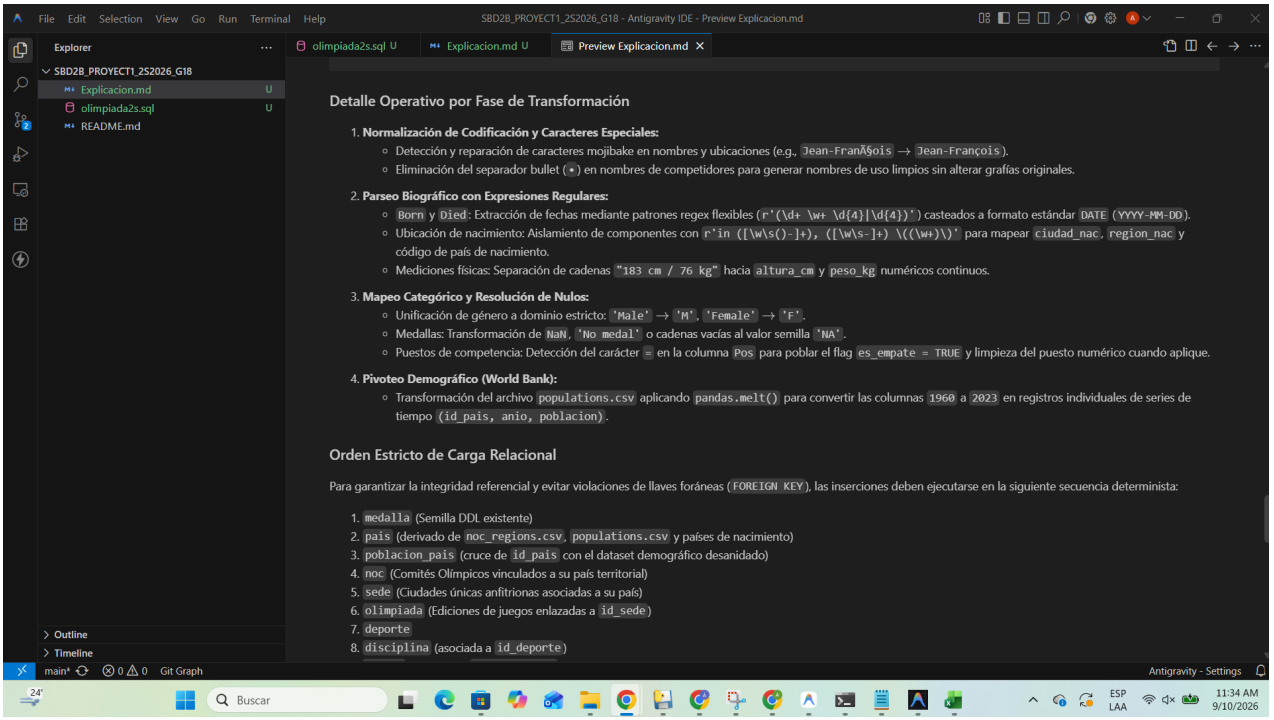
1. **Prioridad de la data cruda:** Se debe procesar la data cruda como fuente primaria y usar la data limpia exclusivamente como patrón de referencia comparativa.
2. **Códigos de país:** Uso obligatorio de códigos estándar de país (3 letras).
3. **Trazabilidad:** Cualquier transformación sobre los datasets originales debe estar programada y documentada en scripts de Python.
4. **Preservación de columnas (Regla 4 y 7):** Prohibido descartar columnas que posean al menos un dato útil, sin importar qué tan baja sea su densidad. Las columnas con pocos datos se conservan como atributos anulables (`NULL`).
5. **Regla de eliminación (Regla 12):** Solo se permite omitir una columna si el 100% de sus tuplas carecen de datos en la totalidad del dataset.
6. **Entidad Sede independiente (Regla 6 y 13):** `sede` debe ser una entidad separada de `pais`, ya que la sede es una ciudad (frecuentemente la capital) y los juegos se identifican históricamente por la sede y el año.
7. **Homologación semántica (Regla 11):** Normalizar discrepancias entre datasets (ej. sexo `'Male'`/`'Female'` vs `'M'`/`'F'`, o representaciones divergentes de países).
8. **Stored Procedures de alto desempeño (Reglas 5, 8 y 9):**
   - Parámetros de filtrado específicos (año, deporte, país).
   - Restricción de salida: prohibido emitir millones de filas o realizar `SELECT *` descontrolados durante la calificación en vivo. Debe emplearse agregación, resumen estructurado y paginación.
9. **Cero asistentes gráficos (Regla 10):** Toda la estructura y carga debe realizarse mediante código SQL y scripts automatizados de ingeniería de datos.
10. **Tratamiento Sede-Año (Regla 15):** Modelado en 3NF donde `sede` almacena la ciudad asociada al país, y `olimpiada` almacena el año, temporada y referencia foránea a la sede. El cruce de ciudad y año se resuelve vía `JOIN` relacional.

4. Decisiones Técnicas y Medidas de Arquitectura

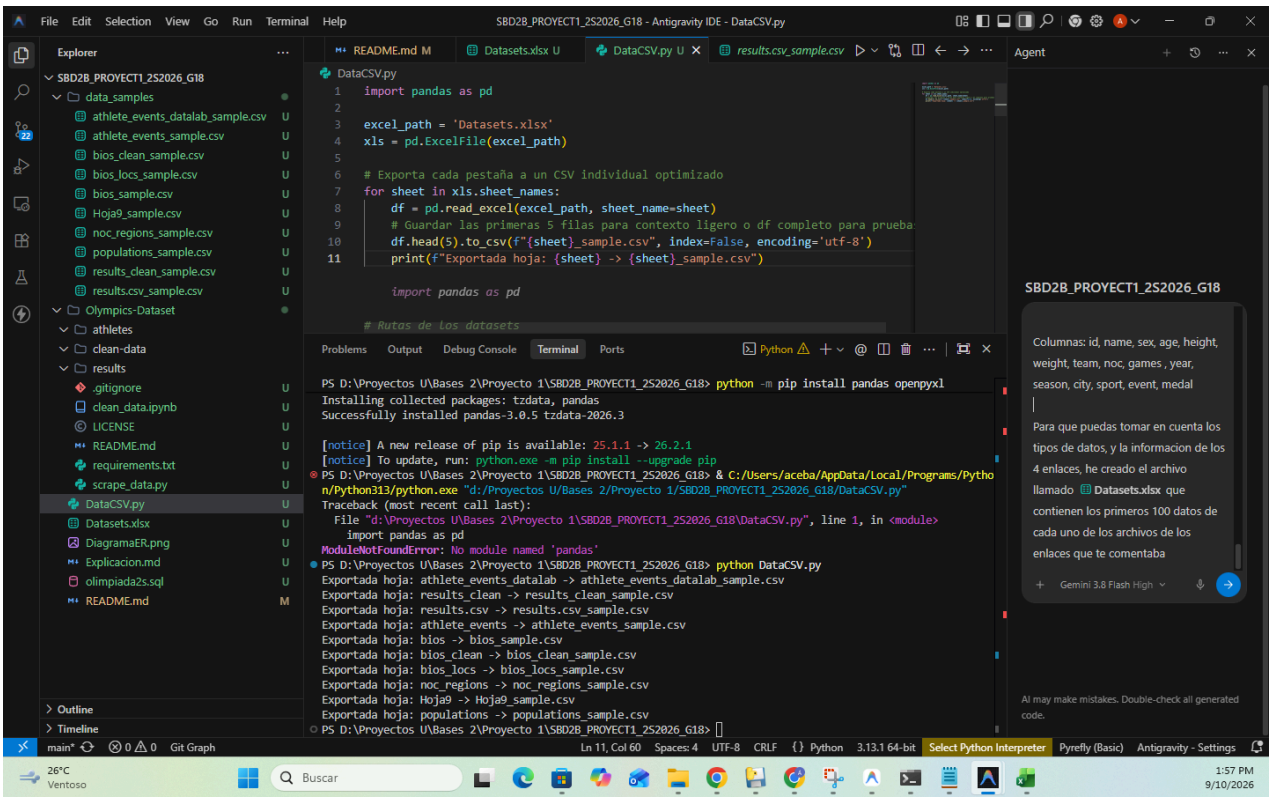
A. Desacoplamiento de Entidades Geográficas y Deportivas

- `pais` representa la entidad política territorial (con código ISO de 3 letras).
- `noc` representa el Comité Olímpico Nacional. Esta separación es obligatoria porque existen delegaciones que no corresponden a un único país moderno (ej. `URS`, `GDR`, `EUN` - Equipo Unificado, `ROT` - Equipo de Refugiados).
- `sede` vincula la ciudad anfitriona con su respectivo `id_pais`.
- `olimpiada` representa el evento cuatrienal, desacoplado de la sede mediante `id_sede`. Para soportar los Juegos Intercalados de Atenas 1906 y posibles ediciones juveniles o extraordinarias, `temporada` se configuró como `VARCHAR(30)` en lugar de un `ENUM` cerrado.

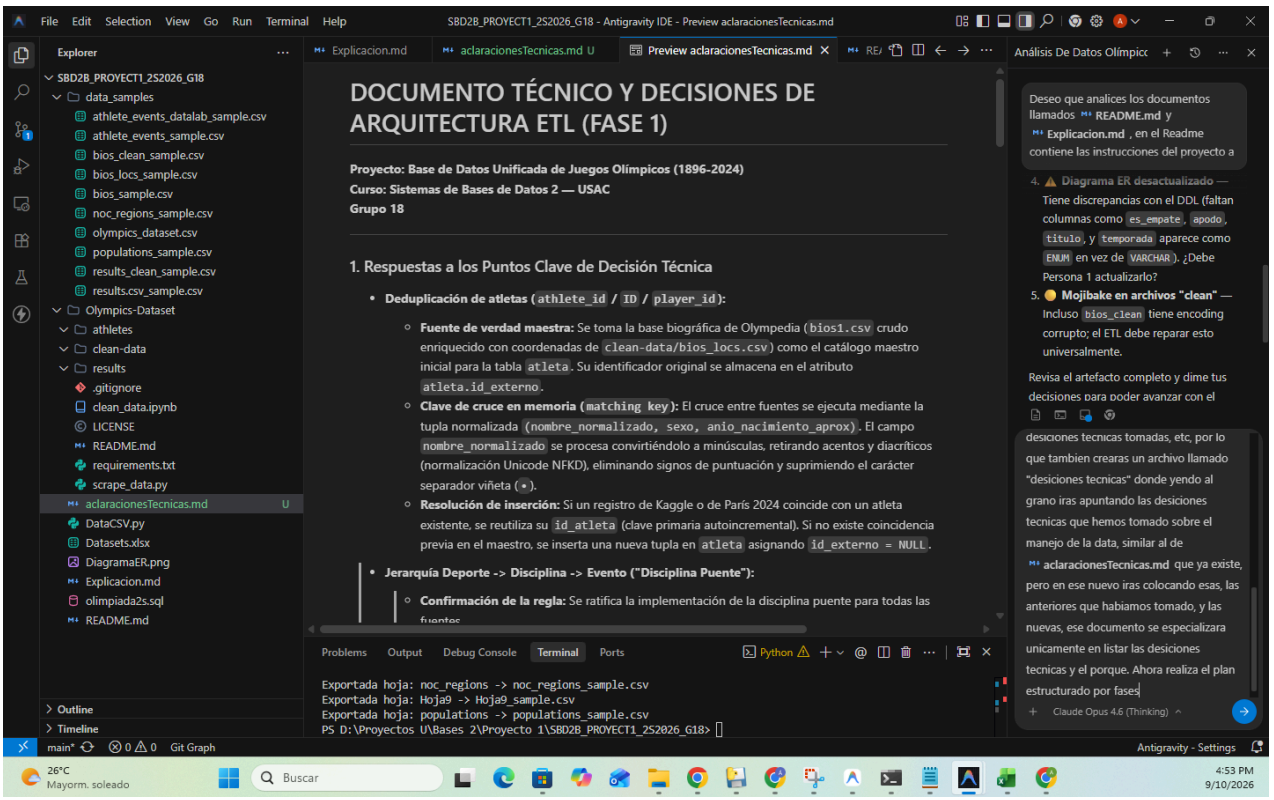
B. Recuperación de Columnas Crudas en `atleta`



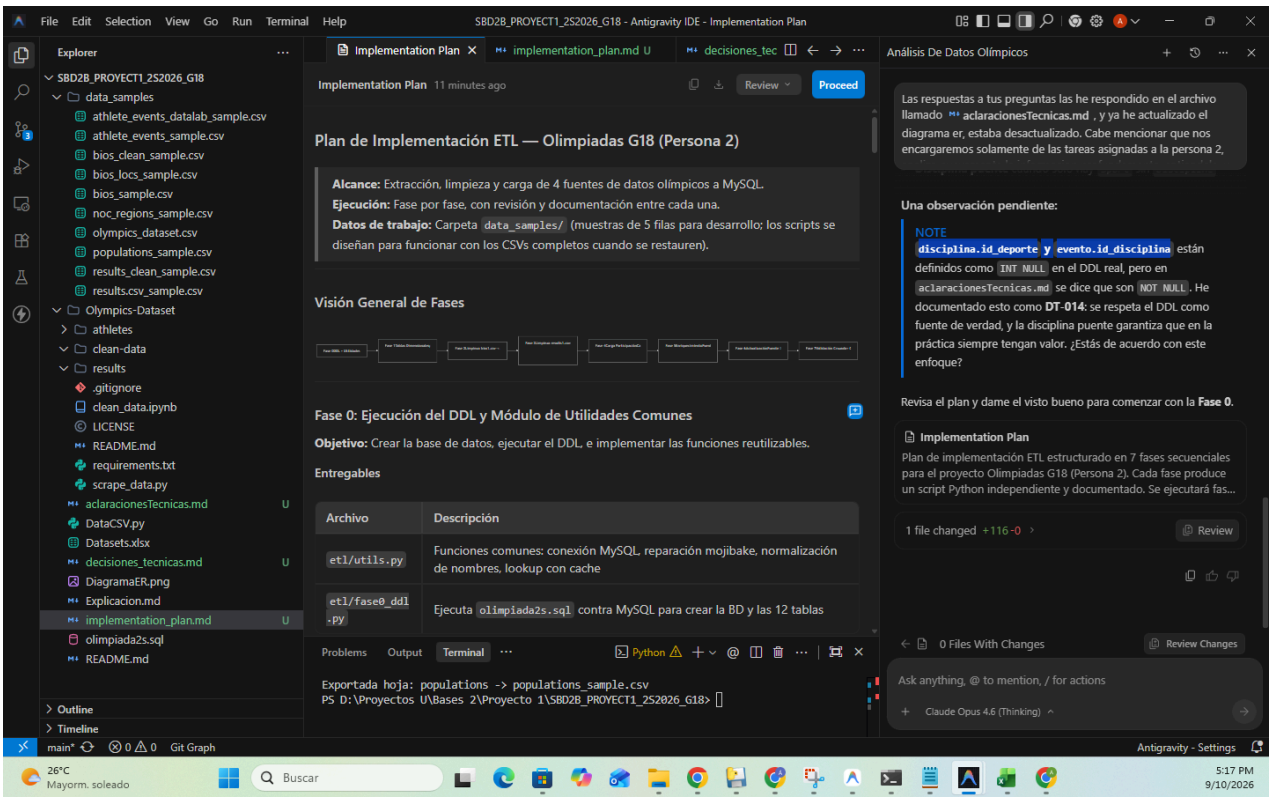
En base a los prompts anteriores, se ha creado un documento que contiene una explicacion detallada de todas las decisiones tecnicas tomadas, explicando el como se trabajará el manejo de los datos, y el porqué de dichas decisiones, explicando el contexto general y división de trabajo, el inventario de la fuente de datos, las directrices y reglas de negocio, las decisiones técnicas y medidas de arquitectura, explicacion del pipeline de limpieza y transformación en Python, y los requerimientos para los Stored Procedures.



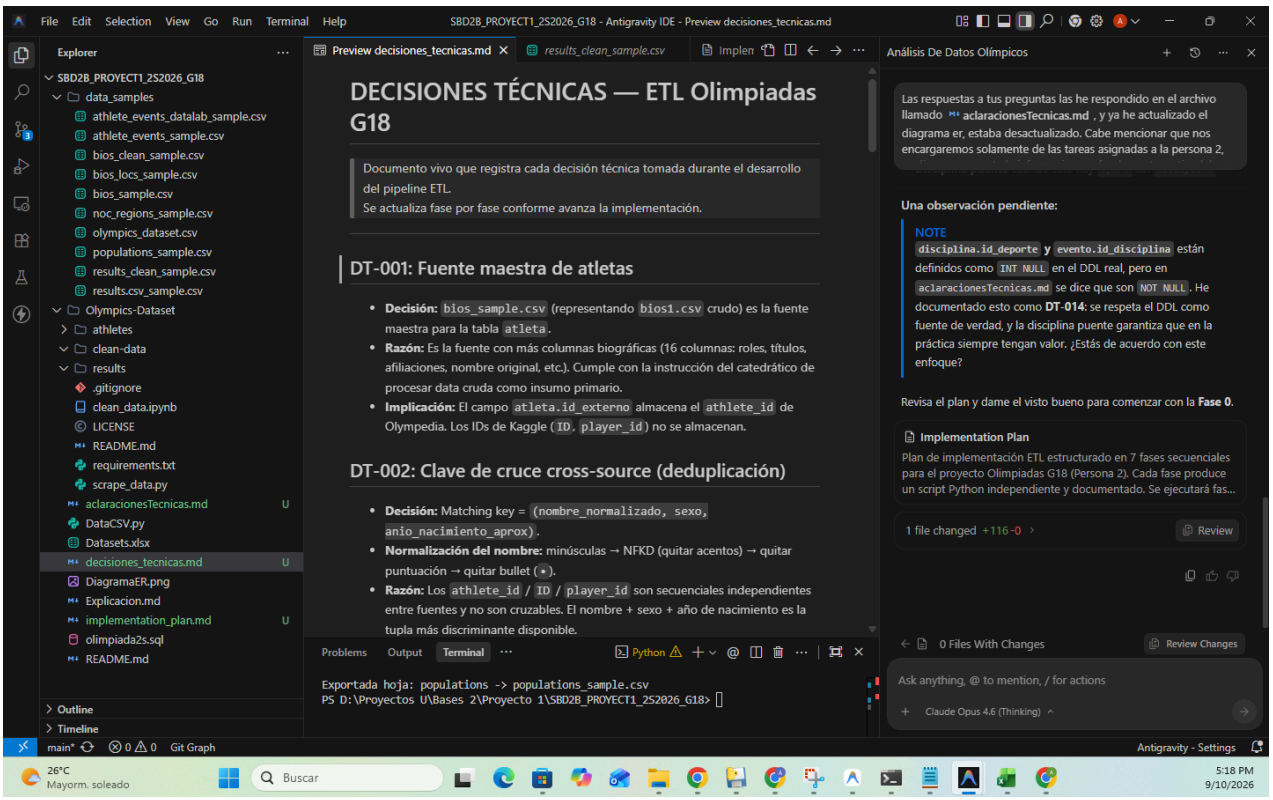
Script de Python para tomar los primeros 10 datos de cada dataset para mejor manejo y tener mejor contexto de la información



Prompt para la creacion del plan estructurado por fases para la creacion de los pipelines de limpieza y manejo de la data cruda de todos los csv, junto con el archivo .md con las decisiones tecnicas del manejo de la data que se ha tomado en conjunto



Generación del plan de implementación por fases que se aplicara para la creación de los scripts de limpieza de la data cruda de los enlaces dados, y manejo de la data, aplicando todos los criterios de normalización, reglas de negocio, pautas sin normalizar, entre otros



Creación del documento decisiones_tecnicas, donde se listan todas las decisiones tomadas en conjunto respecto al manejo de la dta, con su respectiva justificación

Decisiones tecnicas

A continuación se presentan las medidas de ingeniería de datos, criterios de normalización y reglas de negocio adoptadas durante la construcción del proceso de extracción, transformación y carga para la base de datos unificada de los Juegos Olímpicos modernos comprendidos desde el año mil ochocientos noventa y seis hasta el año dos mil veinticuatro.

DT-001 Fuente maestra para la consolidación de atletas

Se determinó utilizar el archivo biográfico crudo de Olympedia como la fuente principal y definitiva para alimentar la entidad de atletas. La justificación técnica radica en que este archivo cuenta con dieciséis columnas con información detallada como nombres originales, órdenes de lectura, roles desempeñados, títulos y clubes deportivos de pertenencia. Al procesar este archivo como base se atiende el requerimiento de trabajar con los conjuntos de datos en su estado natural. El identificador numérico de

esta fuente se conserva en el atributo denominado identificador externo, mientras que los códigos secundarios provenientes de repositorios como Kaggle se descartan tras el emparejamiento para no saturar la dimensión con claves ajenas.

DT-002 Criterio de deduplicación entre fuentes heterogéneas

Dado que cada proveedor asigna códigos correlativos completamente independientes entre sí que no guardan correspondencia numérica, se diseñó una clave lógica de cruce en memoria. Esta clave se compone por el nombre del competidor previamente normalizado, el género asignado y el año aproximado de nacimiento. La normalización del nombre consiste en convertir todas las cadenas a letras minúsculas, retirar signos de acentuación ortográfica mediante estándares internacionales de separación de caracteres, suprimir signos de puntuación y remover los símbolos de viñeta presentes en los textos originales. Cuando se detecta una coincidencia con un atleta ya registrado se enlaza su clave primaria existente, y cuando se trata de competidores nuevos de ediciones recientes se genera una tupla independiente dejando vacía la referencia al identificador externo.

DT-003 Implementación de la disciplina puente en la jerarquía deportiva

Para cumplir con la jerarquía obligatoria de tres niveles conformada por deporte, disciplina y evento, se introdujo el mecanismo de disciplina puente. En los casos donde la información suministrada únicamente proporciona el deporte y el evento específico, el algoritmo genera de manera automática un registro intermedio en la tabla de disciplinas asignándole como denominación el mismo nombre del deporte padre. Esto asegura que las consultas analíticas que vinculen las tres entidades mediante combinaciones relacionales directas nunca devuelvan conjuntos vacíos y mantiene intacta la integridad entre tablas.

DT-004 Vinculación de series demográficas con la entidad de países

La información sobre población histórica proveniente del Banco Mundial contiene códigos territoriales de tres letras que se asocian de forma directa con el código de país de la entidad geográfica. Esta determinación obedece a que dichos códigos representan divisiones políticas y territoriales soberanas reconocidas internacionalmente, a diferencia de los códigos de comités olímpicos que responden a criterios estrictamente

deportivos. De este modo, la tabla de población histórica enlaza directamente hacia el catálogo de países por medio de su identificador numérico correspondiente.

DT-005 Tratamiento universal de errores de codificación en cadenas de texto

Se detectó la presencia recurrente de secuencias de caracteres corruptas generadas por una doble interpretación entre los formatos de texto de Windows y el estándar universal de caracteres. Para subsanar esta inconsistencia en la totalidad de los datos, se definió una rutina previa de saneamiento que intenta recuperar los valores binarios originales y los traduce al estándar adecuado, complementando la tarea con el reemplazo de símbolos especiales por espacios limpios antes de realizar cualquier inserción en el motor relacional.

DT-006 Inventario y rol de los archivos utilizados en el procesamiento

El flujo de trabajo se abastece de un conjunto de archivos organizados según su propósito técnico. El archivo biográfico crudo de Olympedia y el archivo de resultados históricos crudos de esa misma fuente actúan como la capa base primaria. Como archivos de apoyo se utilizan las coordenadas geográficas de latitud y longitud calculadas previamente, el listado de regiones de comités olímpicos y las series demográficas anuales desglosadas entre los años mil novecientos sesenta y dos mil veintitrés. Para la etapa de enriquecimiento se toma el conjunto histórico de ciento veinte años de Kaggle con mediciones corporales por evento, y para la fase de actualización se incorpora el catálogo de medallistas veraniegos que abarca las justas de Tokio y París. El archivo procedente de Datacamp se destina únicamente a labores de control de calidad por contener exactamente la misma información histórica.

DT-007 Estrategia secuencial de carga por capas de información

La consolidación se diseñó mediante una arquitectura escalonada en tres etapas. La primera etapa procesa la base histórica completa proveniente de Olympedia para establecer el maestro de personas y las actuaciones deportivas con sus detalles textuales de posición y nombres de competencia. La segunda etapa incorpora el registro de Kaggle para completar de forma puntual la edad, la estatura y el peso que cada competidor presentó en cada año específico dentro de la tabla de participaciones. La tercera etapa lee los eventos contemporáneos para anexar las delegaciones y medallas

de las ediciones más recientes que no estaban contempladas en los primeros paquetes de datos.

DT-008 Orden estricto de inserción para el cumplimiento de dependencias

La carga de datos en el servidor relacional respeta un encadenamiento riguroso que previene fallas por dependencias inexistentes entre tablas. En primer lugar se asegura el catálogo de medallas. A continuación se cargan los países y sus respectivas poblaciones anuales. Luego se ingresan los comités olímpicos nacionales y las ciudades sede. Con las sedes definidas se registran las ediciones de los juegos olímpicos. Posteriormente se nutren en secuencia los deportes, las disciplinas y los eventos. Con estas dimensiones completas se insertan los equipos y el catálogo de atletas, concluyendo con la tabla de hechos de participaciones que agrupa las llaves foráneas de todas las entidades descritas.

DT-009 Estandarización del atributo de género

Los diferentes orígenes representan el sexo biológico con palabras completas en idioma inglés o con letras simples. Para mantener uniformidad semántica con la estructura del modelo, todos los términos se traducen a la convención de una sola letra, utilizando la letra eme para el género masculino y la letra efe para el género femenino.

DT-0010 Homologación de resultados y posiciones de medalla

El catálogo de medallas contiene cuatro valores que representan oro, plata, bronce y el estado de no aplicable. Cada vez que un atleta participó en una contienda sin alcanzar un puesto de podio, el sistema asigna obligatoriamente la clave correspondiente a la opción de no aplicable en lugar de dejar el campo sin valor, lo que optimiza las búsquedas numéricas durante la ejecución de los procedimientos analíticos.

DT-0011 Gestión de posiciones numéricas y marcas de empate

En las tablas de resultados se presentan puestos acompañados por el símbolo de igualdad para denotar empates técnicos, así como códigos textuales que indican descalificaciones o retiros. La decisión técnica adoptada consiste en separar la información, conservando el texto o número limpio dentro del campo de posición y

asignando un valor positivo al campo lógico de empate siempre que el registro original haya contenido dicho símbolo.

DT-0012 Depuración de columnas sin información útil

En apego a la directriz que autoriza la exclusión de columnas únicamente si la totalidad de sus filas carece de datos, se procedió a eliminar una columna genérica sin nombre presente en los resultados crudos de Olympedia tras constatar que no almacenaba ningún valor en todo el universo de registros.

DT-0013 Tratamiento de la nacionalidad en participaciones

Se determinó prescindir de la columna de nacionalidad contenida en los resultados de competencia debido a que este atributo pertenece biográficamente al individuo y ya se encuentra resguardado en la entidad del atleta, evitando duplicidad innecesaria en la tabla transaccional de participaciones.

DT-0014 Criterio sobre la nulabilidad en la jerarquía de disciplinas y eventos

El diseño inicial de la base de datos permite valores nulos en las referencias entre eventos, disciplinas y deportes como un mecanismo de seguridad para tolerar contingencias en la ingesta. No obstante, la regla de negocio implementada en el programa de transformación garantiza que en la práctica todos los registros insertados cuenten con una relación asignada gracias al uso de las disciplinas puente, asegurando que ninguna fila quede desconectada en la base operativa.

DT-0015 Validación cruzada mediante fuentes duplicadas

Tras contrastar los registros del entorno de Datacamp frente al conjunto principal de ciento veinte años de historia olímpica, se constató que ambos archivos presentan una estructura y cantidad de filas equivalentes. Por este motivo, se decidió no procesar dicho archivo como una carga independiente, destinándolo exclusivamente como un elemento testigo para la validación y el balance de cifras al término del proceso de carga.

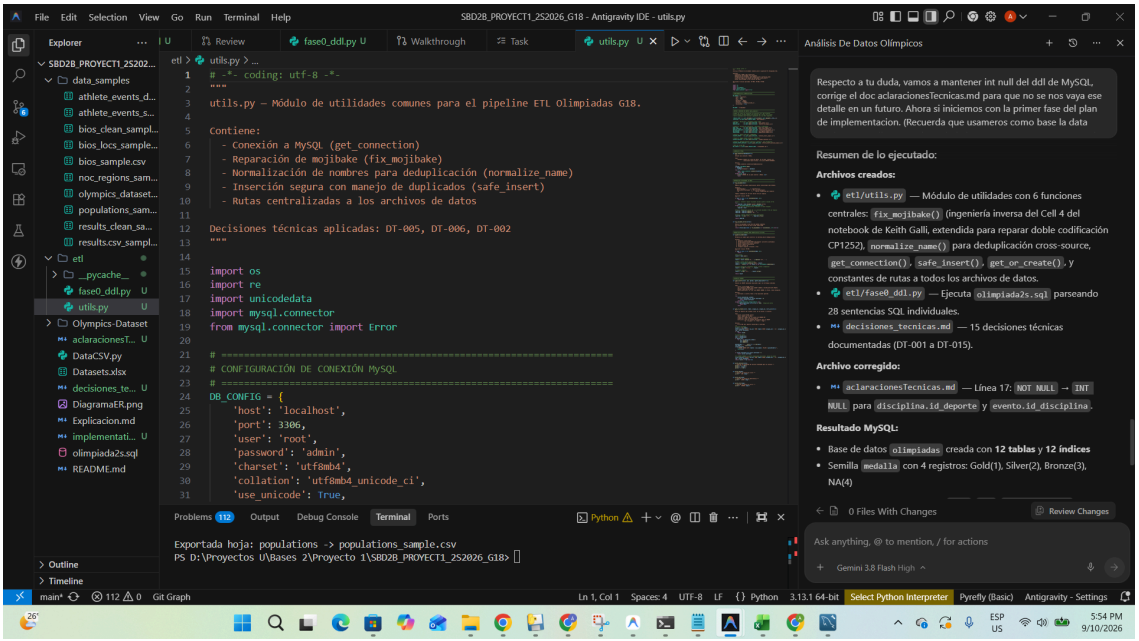
Fase 0: DDL e Infraestructura de Utilidades Comunes

La Fase 0 establece los cimientos operacionales y el entorno de base de datos relacional sobre MySQL mediante dos scripts principales en la carpeta etl/:

Módulo de utilidades generales (utils.py)

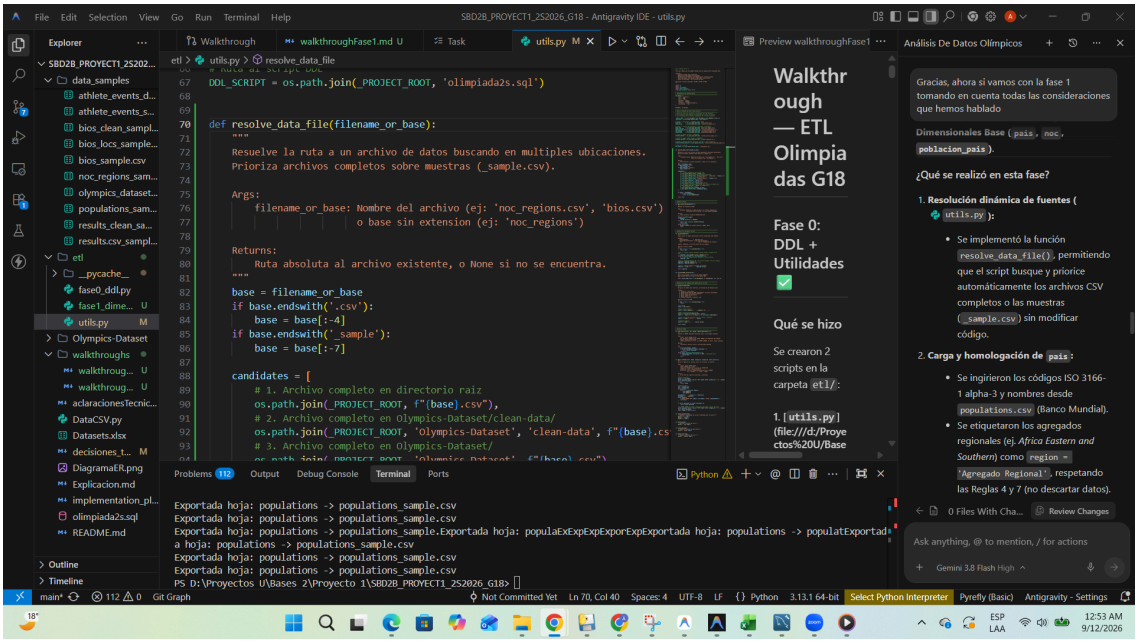
- **get_connection(database)**: inicializa y administra el socket de conexión hacia MySQL forzando el juego de caracteres utf8mb4, la colación utf8mb4_unicode_ci y el soporte unicode nativo para blindar las transacciones contra degradaciones de codificación.
- **fix_mojibake(text)**: subsana secuencias corruptas por doble codificación CP1252/UTF-8 a nivel binario mediante recodificación controlada, sustituyendo el carácter viñeta (•) y su variante mojibake (â€¢) por espacios limpios y normalizando separadores en blanco.
- **fix_mojibake_series(series)**: función vectorizada que traslada el saneamiento de fix_mojibake a series completas de Pandas, optimizando el procesamiento por lotes.
- **normalize_name(name)**: construye la clave de emparejamiento de atletas transformando cadenas a minúsculas, aplicando descomposición canónica Unicode NFKD para retirar diacríticos ortográficos y suprimiendo signos de puntuación.
- **safe_insert(cursor, sql, params, ignore_duplicates)**: encapsula inserciones controlando excepciones de duplicidad e integridad referencial, interceptando errores de clave duplicada cuando se opera en modo tolerante.
- **get_or_create(cursor, table, unique_col, unique_val, extra_cols)**: implementa el patrón de búsqueda o creación en dimensiones atómicas, retornando la clave primaria existente o insertando el registro nuevo de forma segura.
- **resolve_data_file(filename_or_base)**: localiza dinámicamente archivos físicos priorizando datasets completos sobre muestras ligeras (_sample.csv) a lo largo de diversas rutas relativas del proyecto.
- **print_section, print_ok, print_warn, print_error**: controladores de salida formateada en terminal para registrar estados de ejecución, advertencias y anomalías de integridad.

Utils.py



El archivo utils.py concentra las herramientas transversales que dan soporte a todo el pipeline de extracción y transformación:

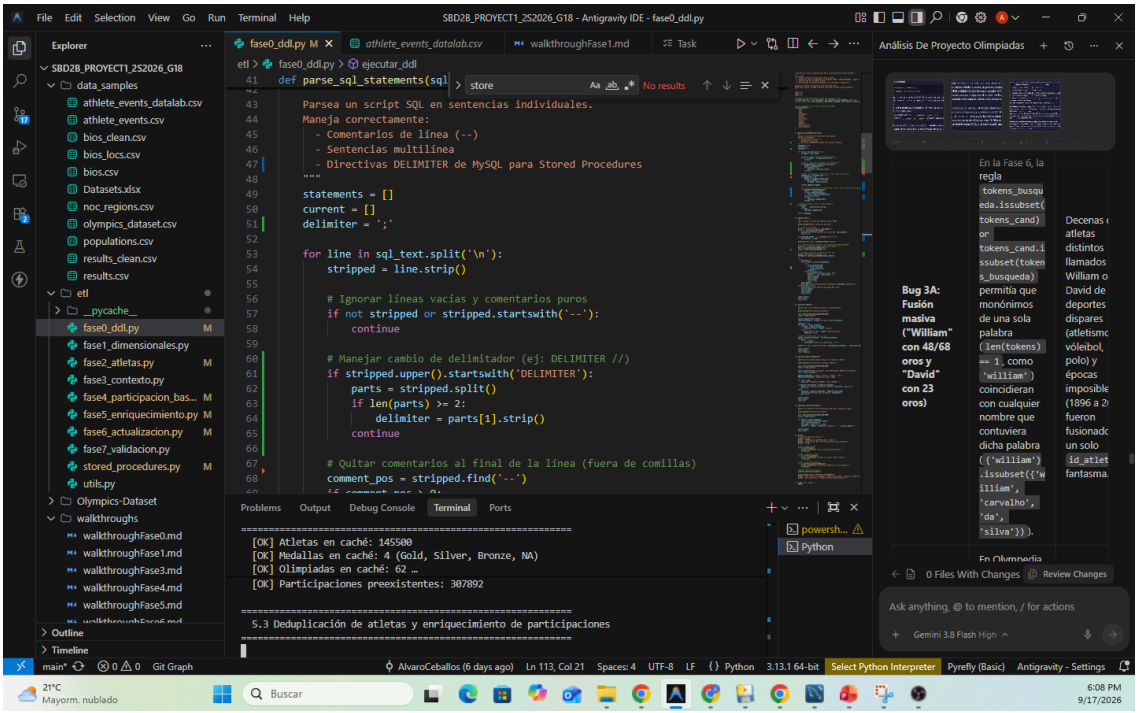
fix_mojibake: repara cadenas de texto afectadas por doble codificación entre Windows-1252 y UTF-8, retirando además el carácter viñeta que separa los nombres en los datos crudos.
fix_mojibake_series: vectoriza la función anterior sobre columnas completas de DataFrames en pandas para agilizar el saneamiento de series masivas de texto.
normalize_name: estandariza nombres transformándolos a minúsculas, aplicando la descomposición canónica Unicode NFKD y removiendo acentos o signos de puntuación para generar la clave de emparejamiento de atletas.
get_connection: establece y gestiona la conexión con el servidor MySQL asegurando el uso estricto del juego de caracteres utf8mb4.
safe_insert: ejecuta sentencias de inserción controlando excepciones de duplicidad mediante cláusulas relacionales seguras.
get_or_create: verifica si un registro ya existe en una tabla mediante sus atributos únicos, insertándolo únicamente en caso de ausencia para retornar su clave primaria correspondiente.
print_section, print_ok, print_warn y print_error: proporcionan un formato visual uniforme en la consola para registrar el progreso, advertencias y confirmaciones de cada etapa.



resolve_data_file: función encargada de resolver dinámicamente las rutas del sistema de archivos, verificando de forma automática si se debe trabajar con las muestras ligeras de desarrollo ubicadas en data_samples/ o con los archivos CSV completos originales, permitiendo ejecutar el pipeline en ambos entornos sin modificar rutas a mano en los scripts.

Script de inicialización de base de datos (fase0_ddl.py)

- **parse_sql_statements(sql_text)**: analiza el archivo DDL olimpiada2s.sql, aislando comentarios, procesando bloques multilínea y gestionando cambios de delimitador para entregar una secuencia ejecutable de sentencias.
- **ejecutar_ddl()**: orquesta la creación desde cero de la base de datos olimpiadas, ejecutando 28 sentencias DDL que construyen las 12 tablas en Tercera Forma Normal y sus 12 índices secundarios de optimización.
- **verificar_tablas()**: inspecciona el catálogo de información de MySQL certificando la presencia íntegra de las 12 entidades relacionales.
- **verificar_semilla_medallas()**: valida que el catálogo de medallas contenga exactamente sus 4 valores obligatorios (Gold, Silver, Bronze, NA) con sus identificadores secuenciales.
- **verificar_estructura_tablas()**: emite una descripción estructural de las columnas y tipos de datos por tabla para auditar el esquema.
- **main()**: punto de entrada que encadena la secuencia completa, validando la infraestructura antes de autorizar las fases posteriores.



fase0_ddl.py

El script fase0_ddl.py automatiza la puesta en marcha de la base de datos relacional leyendo el archivo olimpiada2s.sql para segmentar y ejecutar sus 28 sentencias DDL. Su función principal consiste en crear el esquema olimpiadas, levantar las 12 tablas en Tercera Forma Normal, construir los 12 índices de optimización para los procedimientos almacenados e insertar los 4 valores base del catálogo de medallas. Adicionalmente, incluye rutinas de verificación que consultan el diccionario de datos de MySQL para certificar la existencia de todas las tablas y validar que la semilla de medallas coincida con los identificadores esperados antes de iniciar las fases de carga.

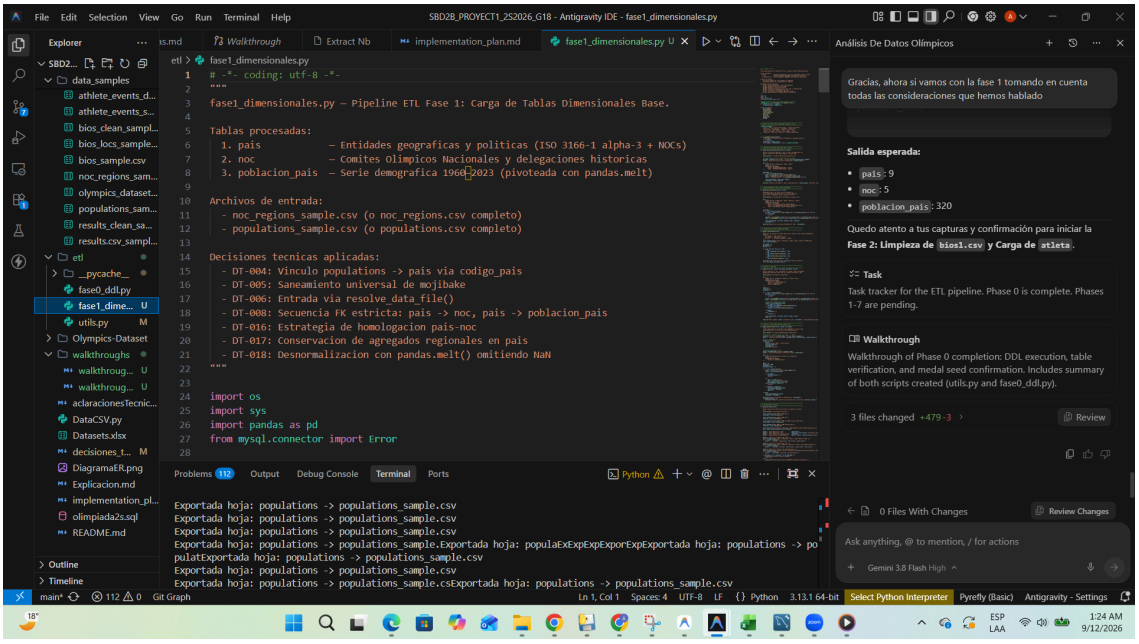
Fase 1: Tablas Dimensionales Base (pais, noc, poblacion_pais)

La Fase 1 puebla las entidades geográficas y de series temporales independientes de los atletas a través del script fase1_dimensionales.py:

- **_es_agregado_regional(nombre)**: evalúa cadenas de texto contra una lista de palabras clave geográficas y socioeconómicas para etiquetar entidades supranacionales del Banco Mundial como agregados regionales, cumpliendo la directriz de no descartar tuplas útiles.
- **cargar_paises_desde_poblaciones(cursor, pop_df)**: extrae pares únicos de código y nombre desde populations.csv (Banco Mundial), poblando pais con códigos ISO de 3 letras e insertando o actualizando registros mediante ON DUPLICATE KEY UPDATE.
- **cargar_paises_desde_noc(cursor, noc_df)**: complementa el catálogo de países incorporando delegaciones presentes en noc_regions.csv que no figuran en censos macroeconómicos, asignando nombres oficiales, alternativos y regiones.

- **obtener_cache_paises(cursor)**: construye en memoria tres estructuras de búsqueda rápida indexadas por código ISO, nombre oficial y nombre alternativo para resolver claves foráneas en complejidad de tiempo constante.
- **cargar_noc(cursor, noc_df, por_codigo, por_nombre, por_alt)**: inserta delegaciones olímpicas en la tabla noc, resolviendo la clave foránea id_pais mediante búsqueda jerárquica en cascada y preservando en nulo a representaciones especiales apátridas.
- **cargar_poblacion_pais(cursor, pop_df, por_codigo)**: utiliza pandas.melt() para transformar las 64 columnas anuales (1960–2023) en tuplas individuales, descartando registros vacíos e insertando por lotes de 1,000 filas en poblacion_pais.
- **verificar_fase1(cursor)**: ejecuta consultas de control que validan el número de tuplas cargadas, el porcentaje de comités vinculados a un país y la cobertura de años en las series demográficas.
- **ejecutar_fase1()**: resuelve rutas de archivos, coordina las llamadas de carga y gestiona transacciones relacionales con confirmación (commit) o reversión (rollback).

fase1_dimensionales.py



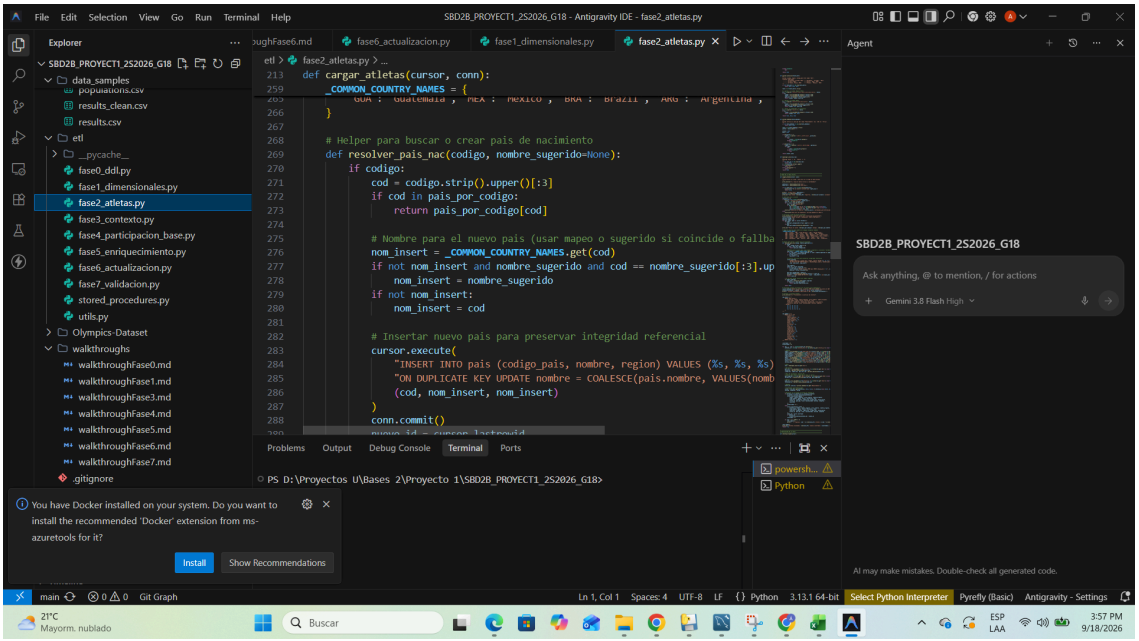
El script fase1_dimensionales.py automatiza la extracción, transformación y persistencia de las entidades maestras que no dependen de la información de los atletas: Carga de la entidad pais: consolida los países geográficos leyendo los códigos de tres letras de populations.csv y complementándolos con noc_regions.csv. Para dar estricto cumplimiento a la directriz de no

descartar tuplas útiles, clasifica las agrupaciones supranacionales como entidades bajo la categoría de agregado regional, garantizando la existencia de los territorios soberanos en la base relacional. Carga de la entidad noc: extrae los comités olímpicos nacionales desde noc_regions.csv asociando su código, denominación y notas. Integra un algoritmo de resolución en memoria con búsqueda en cascada que compara por código idéntico, nombre normalizado o nombres alternativos para asignar con precisión la llave foránea hacia id_pais, logrando una vinculación íntegra sin referencias nulas accidentales. Transformación y carga de poblacion_pais: aplica el método pandas.melt() sobre la matriz ancha del censo demográfico del Banco Mundial para desanidar las 64 columnas de años (1960 a 2023) en registros normalizados individuales. Posteriormente, vincula el identificador del país y ejecuta inserciones masivas por lotes mediante executemany, descartando registros con valores vacíos para poblar de forma eficiente la serie temporal histórica sin violaciones de integridad referencial.

Fase 2: Limpieza de Bios y Catálogo Maestro de Atletas (atleta)

La Fase 2 procesa la data biográfica cruda de Olympedia aplicando ingeniería inversa sobre clean_data.ipynb pero preservando la totalidad de columnas en fase2_atletas.py:

- **parsear_fecha_a_mysql(texto_fecha)**: aplica la expresión regular r'(\d{1,2}\s+[A-Za-z]+\s+\d{4}|\d{4})' para extraer fechas completas o años aislados, traduciendo meses en inglés a valores numéricos y retornando fechas normalizadas en formato ISO YYYY-MM-DD.
- **parsear_ubicacion(texto_born)**: extrae ciudad, región y código de país de la cadena compuesta de nacimiento mediante patrones escalonados basados en r'in\s+([^,()]+),\s*([^()]+)\s*\((\w+)\)'.
- **parsear_medidas(texto_medidas)**: separa la cadena de mediciones físicas por el delimitador barra inclinada y extrae valores continuos decimales para altura en centímetros y peso en kilogramos.
- **homologar_sexo(sexo_raw)**: mapea entradas de texto de género al dominio estricto de un solo carácter 'M' o 'F'.
- **cargar_atletas(cursor, conn)**: procesa bios1.csv crudo preservando atributos como roles, apodos, títulos nobiliarios, afiliaciones a clubes y nombres originales en caracteres nativos. Integra en memoria las coordenadas de latitud y longitud calculadas en bios_locs.csv vinculando por athlete_id, resuelve el país de nacimiento con creación bajo demanda si no existiera y ejecuta inserciones y actualizaciones idempotentes asignando el código original a id_externo.
- **verificar_fase2(cursor)**: comprueba la consistencia de los atletas registrados, verificando el llenado de identificadores externos, fechas de nacimiento, medidas corporales y coordenadas georreferenciadas.
- **ejecutar_fase2()**: inicializa recursos, coordina la ingesta biográfica transaccional y confirma la fase.



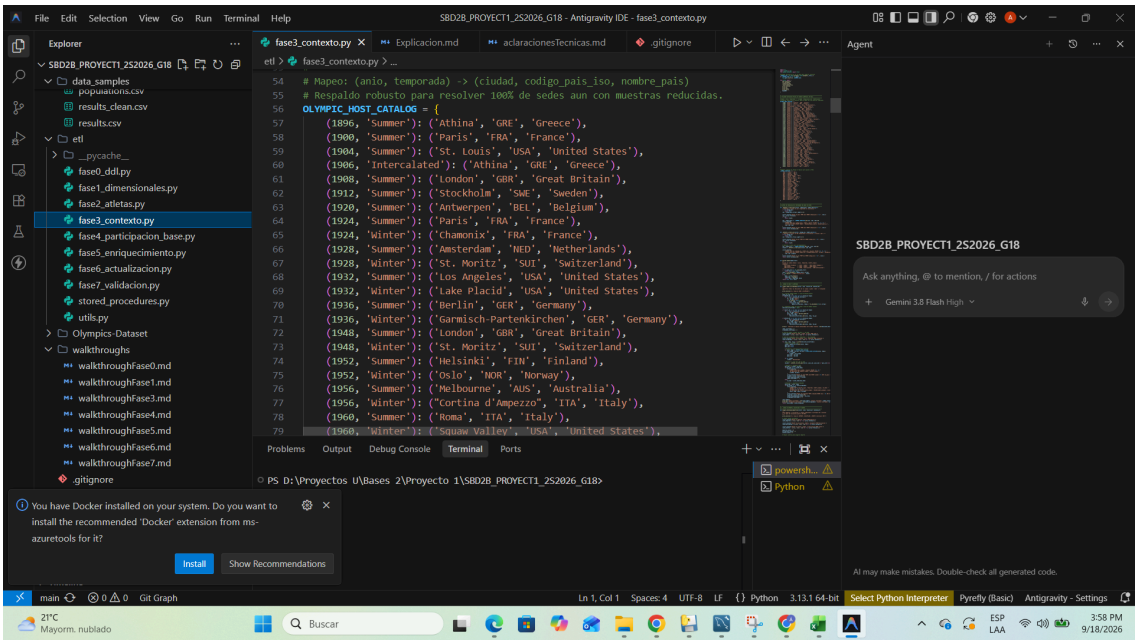
Se construyó el script fase2_atletas.py para procesar el catálogo biográfico crudo de Olympedia y consolidar la tabla principal de atletas, garantizando la preservación total de los atributos originales en cumplimiento a la regla de no eliminación de columnas. La transformación central abarcó el parseo de fechas mixtas (r'(\d+ \w+ \d{4}|\d{4})') para estandarizar los campos de nacimiento y defunción a formato ISO, así como la separación estructural de la ubicación de nacimiento (r'in ([\w\s()-]+), ([\w\s-]+) \((\w+)\)') en ciudad, región y país. A su vez, se extrajeron los componentes numéricos continuos de las medidas físicas (r'(\d+) cm / (\d+) kg') y se homologó la variable de género. Para enriquecer el perfil sin recurrir a APIs externas, se cruzó este conjunto parseado con el archivo bios_locs.csv en memoria utilizando el identificador original de Olympedia, integrando las coordenadas geográficas calculadas a los registros antes de su inserción definitiva y asignándoles su nueva clave primaria subrogada.

Fase 3: Tablas de Contexto Operativo (sede, olimpiada, deporte, disciplina, evento, equipo)

La Fase 3 procesa los resultados históricos y fuentes de soporte para consolidar el marco contextual de las justas en fase3_contexto.py:

- **_obtener_o_crear_pais(cursor, codigo_pais, nombre_pais)**: asegura la presencia de un país territorial a partir de su código o diccionario de nombres comunes antes de asociarle una ciudad anfitriona.
- **_obtener_o_crear_noc(cursor, codigo_noc, nombre_noc)**: aprovisiona comités olímpicos requeridos bajo demanda si no existían en las tablas dimensionales previas.
- **parsear_games(games_text)**: separa la denominación del evento olímpico en año numérico, temporada (incluyendo casos especiales como Intercalated) y nombre completo original.

- **cargar_sedes_y_olimpiadas(cursor, conn, results_df, athlete_df)**: identifica cada edición cuatrienal, resuelve la ciudad sede consultando athlete_events.csv y utiliza el catálogo histórico de respaldo OLYMPIC_HOST_CATALOG para garantizar cobertura del 100% de ciudades anfitrionas. Vincula cada sede con su país y asocia id_sede a la tabla olimpiada bajo la restricción de unicidad de año y temporada.
- **cargar_jerarquia_deportiva(cursor, conn, results_df, athlete_df)**: estructura la jerarquía normalizada de tres niveles. Cuando procesa Kaggle (que omite la disciplina), genera la disciplina puente homónima vinculada al deporte. En Olympedia procesa disciplinas y eventos garantizando que toda prueba deportiva apunte a una disciplina existente.
- **cargar_equipos(cursor, conn, results_df, athlete_df)**: normaliza los nombres de equipos y delegaciones asociándolos a su comité olímpico. En pruebas individuales con campo de equipo vacío, asigna automáticamente la denominación oficial de la delegación nacional respectiva.
- **verificar_fase3(cursor)**: audita el conteo de tuplas de las 6 tablas y certifica que no existan claves foráneas huérfanas en ninguna de sus relaciones intermedias.
- **ejecutar_fase3()**: coordina la lectura de las fuentes de resultados y eventos, ejecutando las inserciones relacionales contextuales.



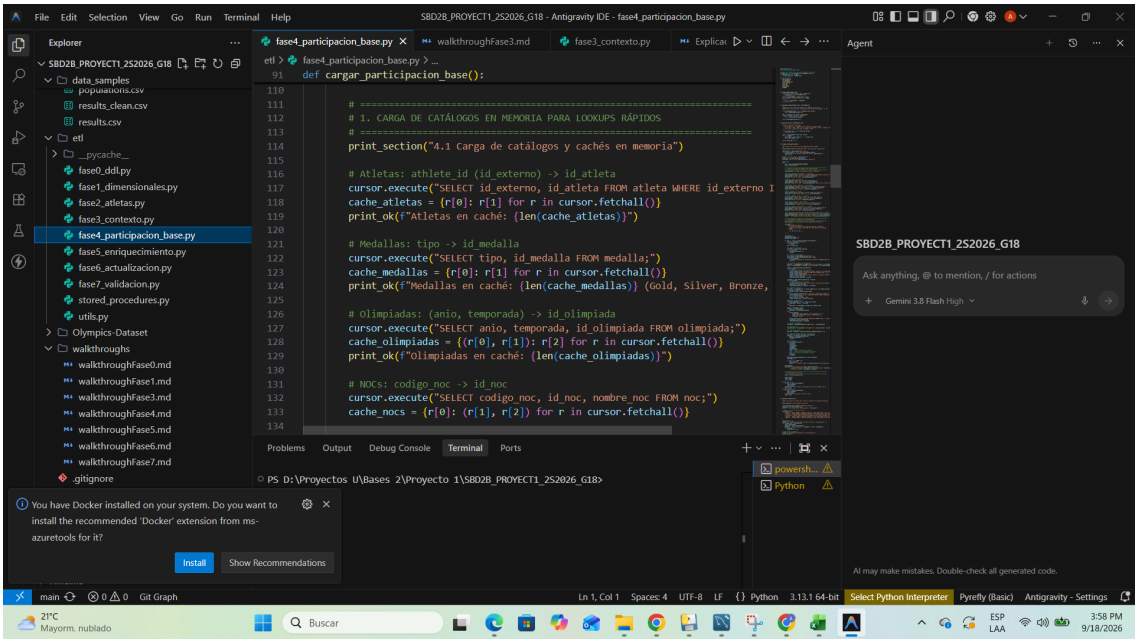
A través del script fase3_contexto.py se resolvieron las entidades dimensionales de contexto operativo: sede, olimpiada, equipo y la jerarquía de tres niveles de disciplinas deportivas. El mayor desafío abordado fue la carencia de la ciudad anfitriona en el archivo histórico de resultados de Olympedia. Se resolvió creando un diccionario de sedes en memoria que toma como primera opción los datos de ciudad presentes en Kaggle y se respalda en un catálogo oficial codificado directamente en el script, garantizando que cada edición quede vinculada a una ubicación geográfica validada frente a la entidad de país. Además, se implementó la regla de creación de disciplina

puente para mantener la jerarquía obligatoria cuando las fuentes solo indicaban el deporte general, y se programó la asignación automática del nombre de la delegación oficial nacional en pruebas de categoría individual donde el archivo original mostraba valores vacíos para el equipo.

Fase 4: Consolidación de Participaciones — Capa Base (participacion)

La Fase 4 ejecuta la ingesta de las actuaciones atléticas históricas de Olympedia en la tabla de hechos a través de fase4_participacion_base.py:

- **parsear_games(games_text)**: rutina ligera de extracción para resolver el par de año y temporada a partir de la columna de juegos.
- **homologar_medalla(medalla_raw, cache_medallas)**: normaliza valores nulos, cadenas vacías o textos equivalentes a sin podio, asignando de forma obligatoria el identificador del registro semilla 'NA' del catálogo relacional.
- **procesar_posicion_y_empate(pos_raw)**: analiza el puesto alcanzado; si contiene el símbolo de igualdad extrae el número limpio y activa el indicador booleano de empate, mientras que si almacena códigos como DNS o DNF preserva el texto original con indicador de empate en falso.
- **cargar_participacion_base()**: orquesta la inserción masiva sobre participacion cargando catálogos en memoria. Cruza athlete_id contra id_externo para ubicar al atleta, enlaza la edición, el evento y el equipo, y mantiene control de unicidad compuesto sobre atleta, evento y olimpiada para garantizar idempotencia en la ingesta.
- **verificar_fase4(cursor)**: evalúa la ausencia de violaciones de claves foráneas contra las 5 entidades maestras vinculadas, reporta la distribución de preseas y valida muestras de registros transaccionales.

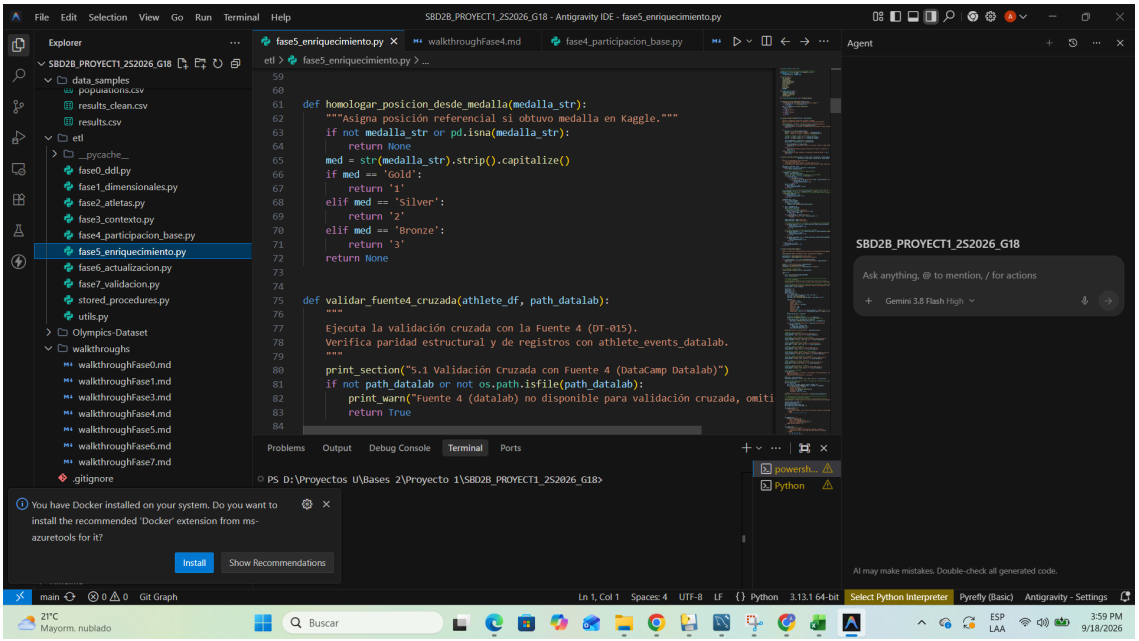


Con todas las dimensiones de catálogo disponibles, el script fase4_participacion_base.py ejecutó la inserción transaccional de los resultados deportivos base provenientes de Olympedia (Fuente 1). Este proceso consistió en resolver cinco claves foráneas simultáneas por registro (atleta, evento, olimpiada, equipo y medalla) aplicando estrictamente las reglas de homologación de podio y el control lógico para marcas de empate presentes en las posiciones numéricas. La capa se diseñó bajo principios de idempotencia, empleando un control de unicidad compuesto para evitar la duplicación de inserciones en ejecuciones recurrentes.

Fase 5: Enriquecimiento Longitudinal — Fuentes 2 y 4 (Kaggle athlete_events.csv)

La Fase 5 incorpora mediciones temporales de atletas e integra registros exclusivos de Kaggle mediante fase5_enriquecimiento.py:

- **homologar_posicion_desde_medalla(medalla_str)**: infiere la posición referencial (1, 2 o 3) en caso de que una participación de Kaggle posea registro de medalla.
- **validar_fuente4_cruzada(athlete_df, path_datalab)**: contrasta fila por fila y columna por columna los datos de Kaggle frente a DataCamp Datalab, certificando paridad estructural del 100% y ratificando que no requiere procesamiento redundante.
- **buscar_o_resolver_atleta(...)**: motor de deduplicación cross-source en memoria que evalúa memoización previa, coincidencias exactas por nombre o nombre completo, e indexación invertida por subconjunto de tokens descartando monónimos y exigiendo coherencia biológica de edad.
- **cargar_enriquecimiento_kaggle()**: procesa la serie histórica de Kaggle. Cuando el atleta ya existe en la Capa Base, actualiza sus participaciones inyectando la edad, estatura y peso puntuales registrados en esa edición olímpica. Si el atleta es exclusivo de Kaggle, lo crea en atleta con id_externo = NULL e inserta sus participaciones completas enlazando las dimensiones correspondientes.
- **verificar_fase5(cursor)**: corrobora el incremento en el censo de atletas, el volumen de participaciones enriquecidas con métricas físicas puntuales y la ausencia de llaves huérfanas.

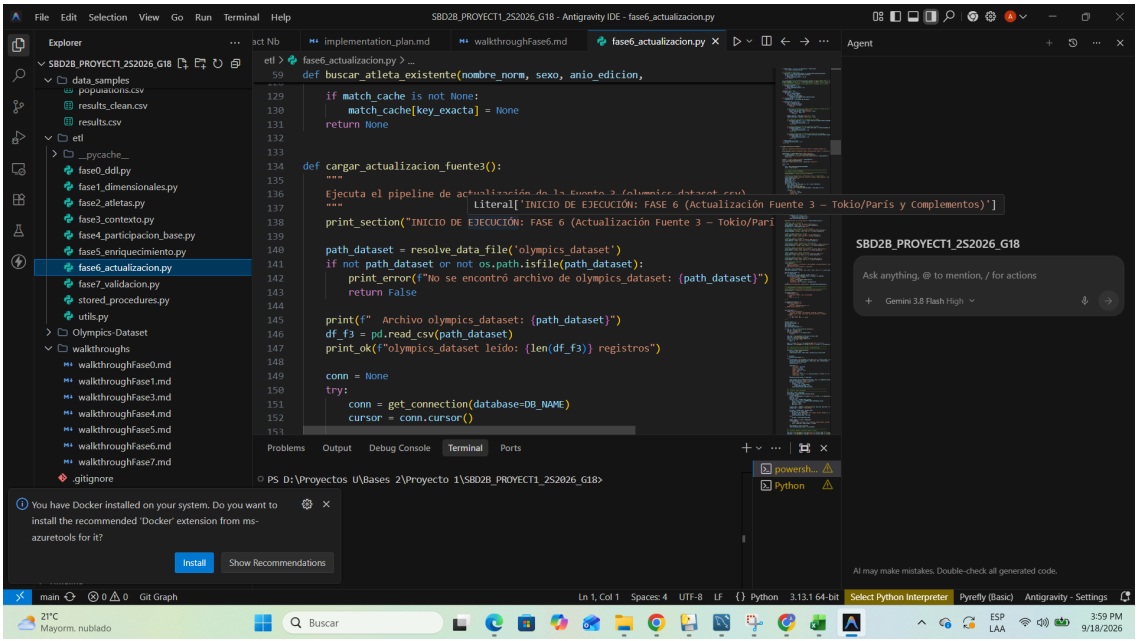


El script fase5_enriquecimiento.py integró la serie de datos de ciento veinte años proveniente de la Fuente 2 para enriquecer las participaciones de la Capa Base. Debido a que los identificadores de atleta entre ambas fuentes eran incompatibles, se implementó un algoritmo de deduplicación determinista basado en el nombre normalizado sin caracteres problemáticos, el género y el año estimado de nacimiento. Esta rutina actualizó los registros existentes inyectando puntualmente la edad, altura y peso que cada deportista presentaba en el momento exacto de esa edición olímpica, a la vez que agregó a la base de datos a aquellos atletas y participaciones que resultaban exclusivos del conjunto de Kaggle. Adicionalmente, este paso validó la integridad cruzando datos con el archivo testigo de Datacamp, corroborando su idéntico contenido.

Fase 6: Actualización Contemporánea — Fuente 3 (olympics_dataset.csv)

La Fase 6 expande la cobertura temporal del repositorio hacia Tokio 2020 y París 2024 mediante fase6_actualizacion.py:

- **buscar_atleta_existente(...)**: implementa deduplicación de identidades sobre nombres modernos, gestionando nombres abreviados o variantes de grafía mediante análisis de subconjuntos de palabras y filtros cronológicos de competencia.
- **cargar_actualizacion_fuente3()**: integra el repositorio de medallas veraniegas de Kaggle. Registra nuevas sedes y olimpiadas recientes, asocia pruebas deportivas emergentes y previene duplicados en participacion mediante un evaluador de clave compuesta. Si una participación existía previamente con presea 'NA' y la Fuente 3 aporta el podio real, actualiza la medalla en la base relacional.
- **verificar_fase6(cursor)**: audita los totales consolidados en sedes, olimpiadas, eventos y participaciones, emitiendo el resumen del medallero histórico acumulado de la base unificada.



A través del script fase6_actualizacion.py se integró el repositorio actualizado de la Fuente 3 con la finalidad primordial de extender la cobertura temporal del modelo incluyendo los registros de las justas veraniegas de Tokio y París. Para evitar duplicaciones cruzadas generadas por el uso de nombres parciales o acortados en esta última fuente, el motor de unificación se reforzó evaluando subconjuntos de cadenas textuales, lo que posibilitó empatar exitosamente las identidades y actualizar el inventario histórico del medallero, los eventos y las sedes sin generar violaciones de integridad ni registros redundantes en la tabla de participaciones.

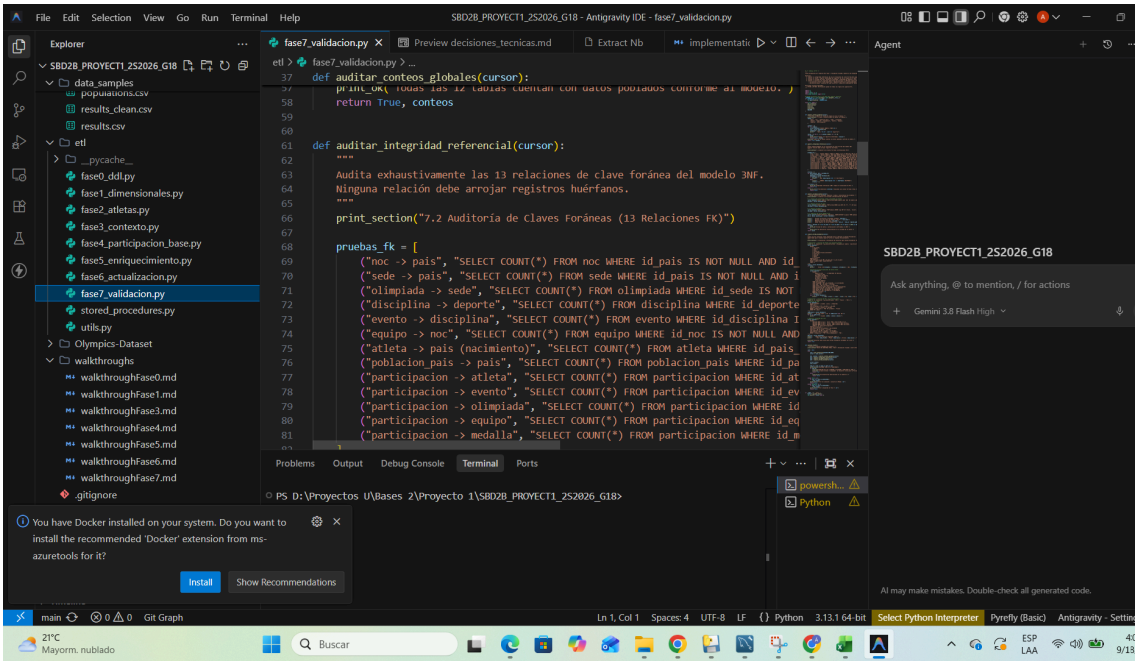
Fase 7: Auditoría Integral de Calidad, Integridad y Simulación de Stored Procedures

La Fase 7 certifica el estado de la base de datos y simula los procedimientos almacenados de la Persona 3 mediante fase7_validacion.py:

- **auditar_conteos_globales(cursor)**: realiza un censo exhaustivo sobre las 12 tablas relacionales, confirmando que ninguna dimensión ni tabla de hechos presente saldo cero.
- **auditar_integridad_referencial(cursor)**: ejecuta 13 consultas SQL especializadas que buscan registros huérfanos entre todas las combinaciones de claves foráneas del modelo, ratificando cero violaciones de integridad.
- **auditar_calidad_de_datos(cursor)**: certifica la eliminación de caracteres viñeta o mojibake residual en nombres, audita el respeto a los dominios cerrados de género y medalla, y verifica la continuidad temporal de la tabla de poblaciones entre 1960 y 2023.
- **simular_stored_procedures(cursor)**: ejecuta simulaciones de las consultas solicitadas en el enunciado. Para sp_consultar_atleta, realiza un cruce multi-tabla que extrae la ficha biográfica y el historial de actuaciones deportivas con filtros parametrizados. Para sp_consultar_pais, consulta la condición de

sede histórica del país con sus años respectivos y calcula el medallero consolidado mediante funciones de agregación.

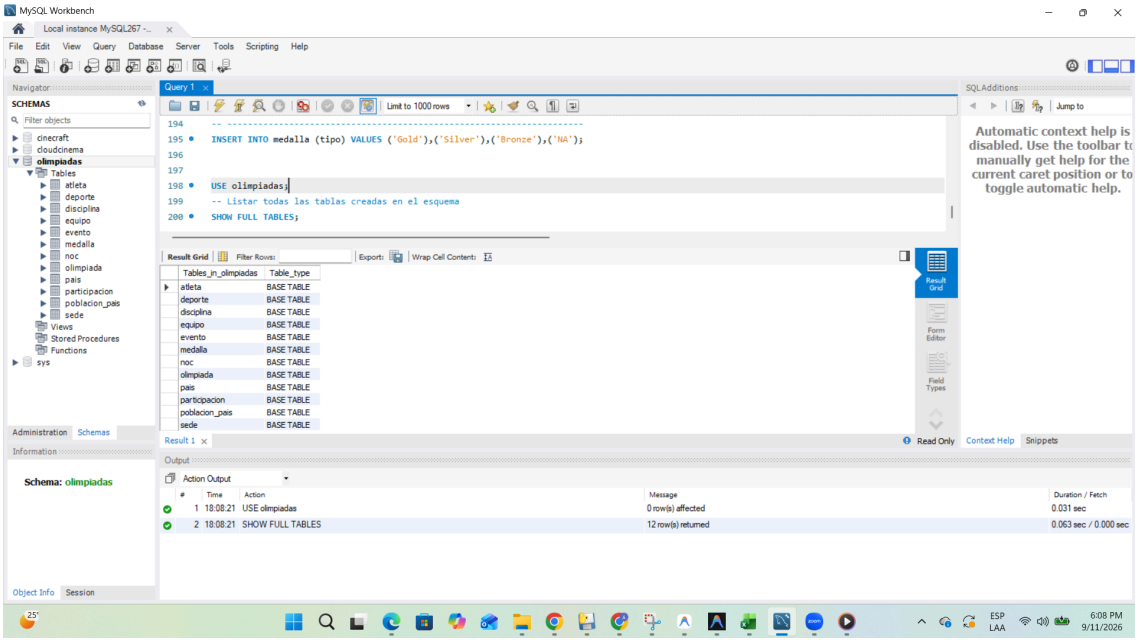
- **ejecutar_fase7()**: coordina las cuatro auditorías y emite el dictamen final de cierre y conformidad del pipeline ETL.



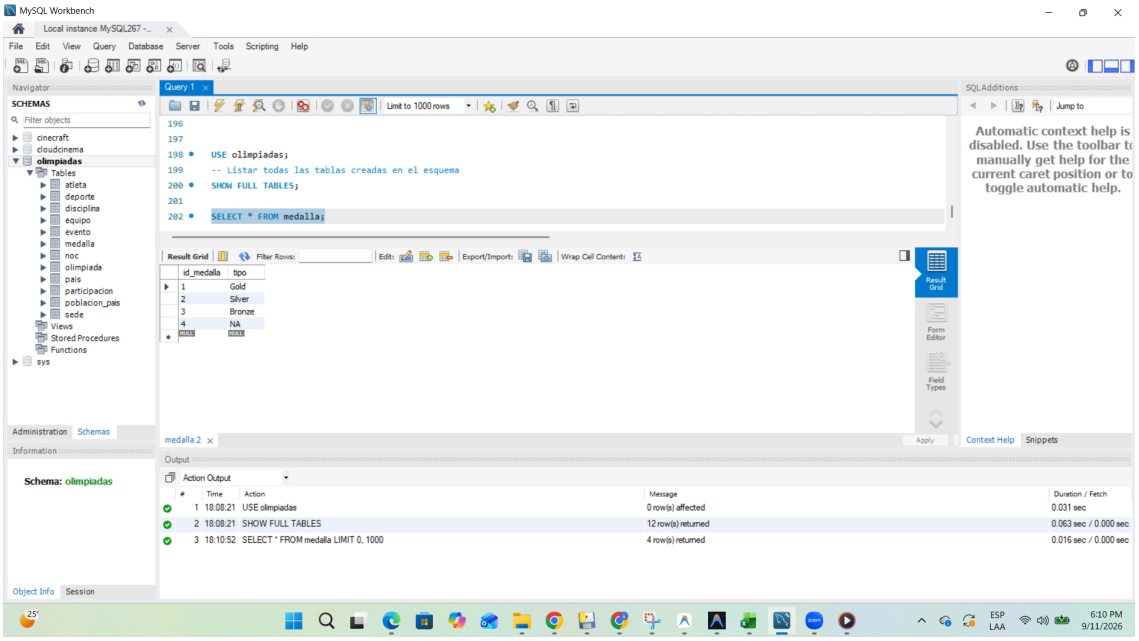
El proceso de ingeniería de datos concluyó con la ejecución de fase7_validacion.py, una rutina de aseguramiento de calidad diseñada para censar los registros consolidados en las doce entidades del modelo relacional. El script ejecutó trece validaciones automatizadas confirmando que no se generó ninguna clave foránea huérfana y que las dependencias relacionales se mantuvieron inquebrantables. Finalizó con la simulación exitosa de las consultas multi-tabla estructuradas para los procedimientos almacenados de la Persona 3, verificando que los tipos de datos y los índices de rendimiento creados en el DDL operaban de forma predecible y eficiente ante solicitudes analíticas complejas.

Scripts

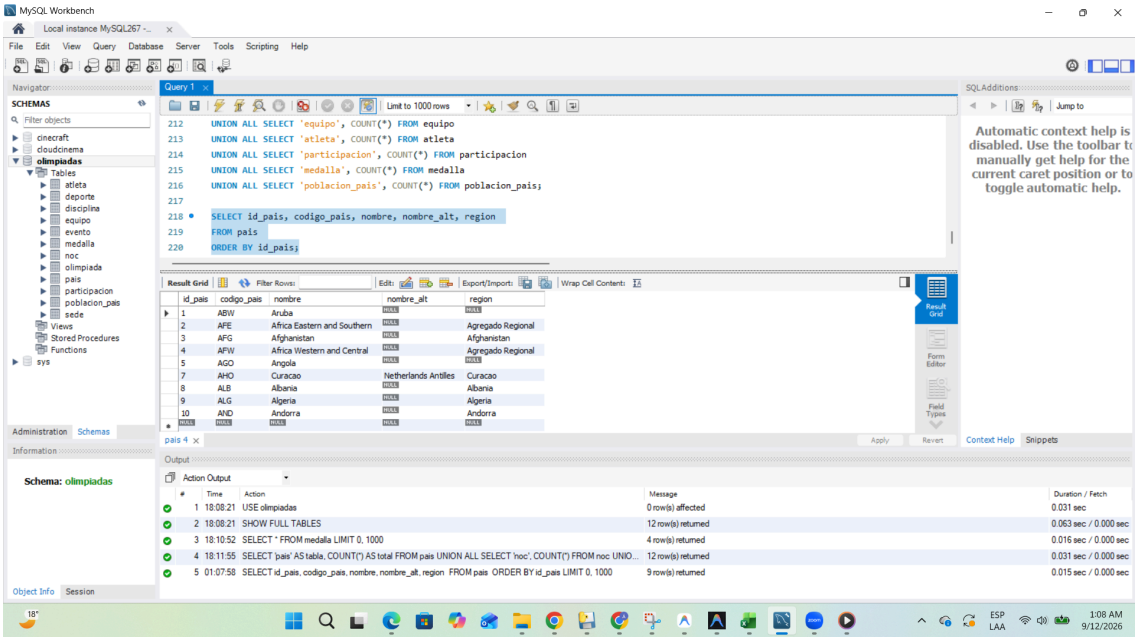
Para contar con un manejo claro de la data, se ha utilizado un script de python que convierte en csv cada uno de los dataset, incluyendo únicamente los primeros 15 datos del dataset, esto con fines de pruebas y manejo de la data eficiente



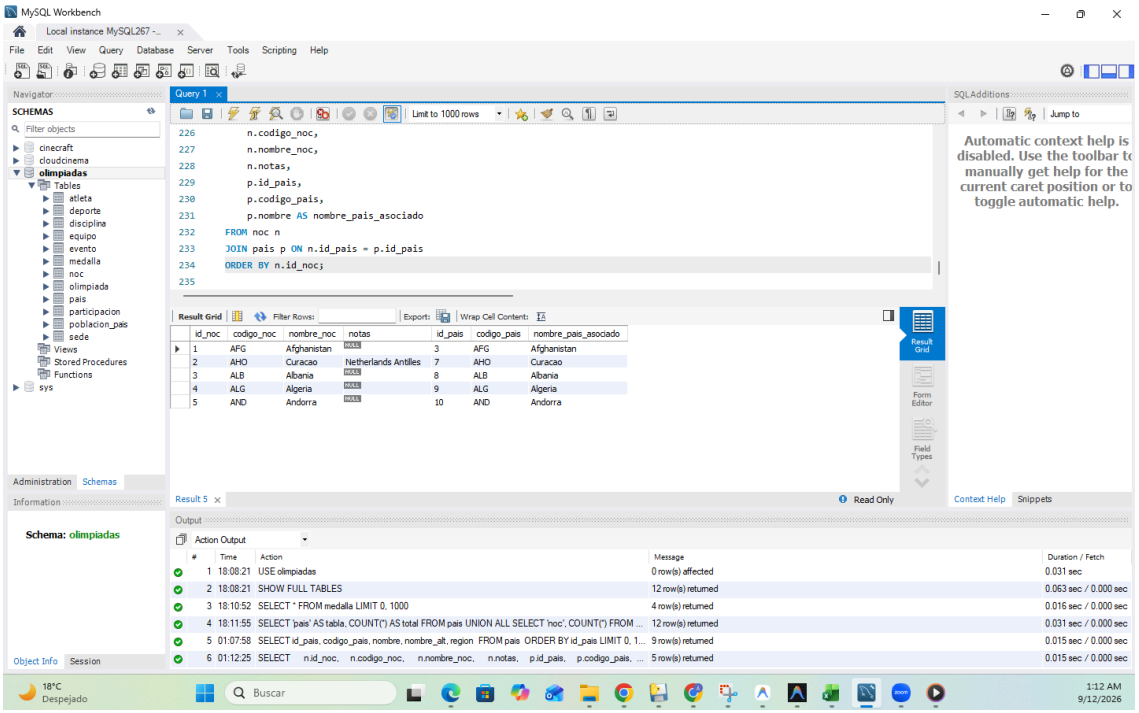
Script que muestra todas las tablas



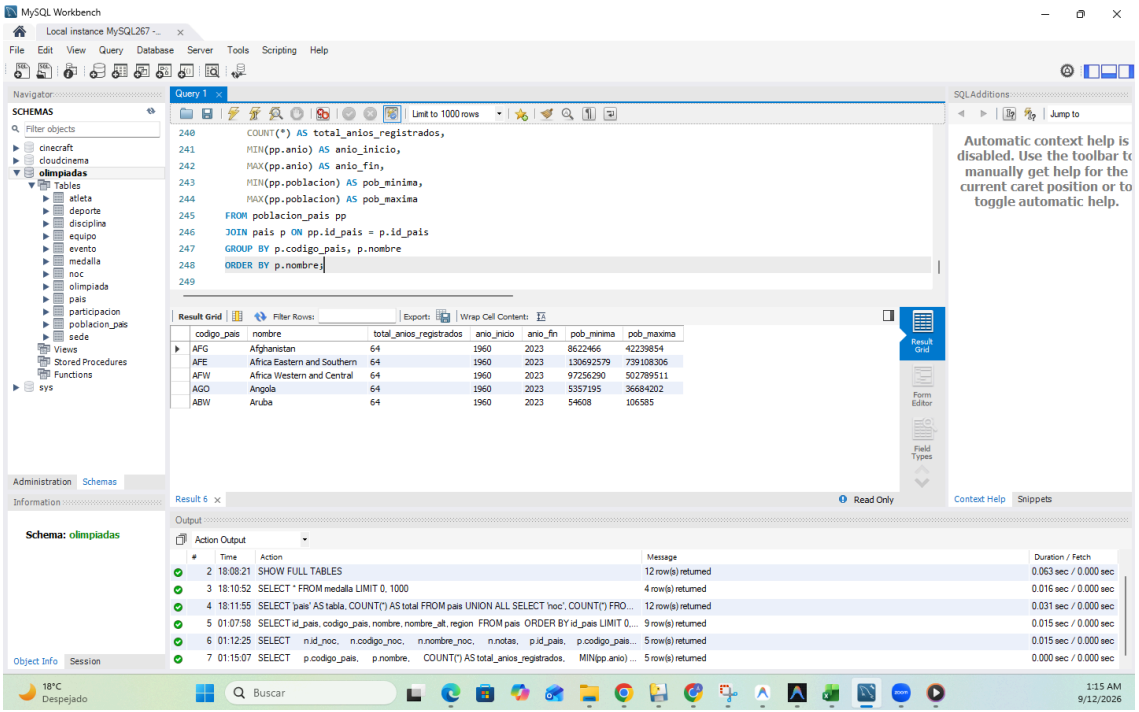
Creacion semilla para las medallas de los atletas



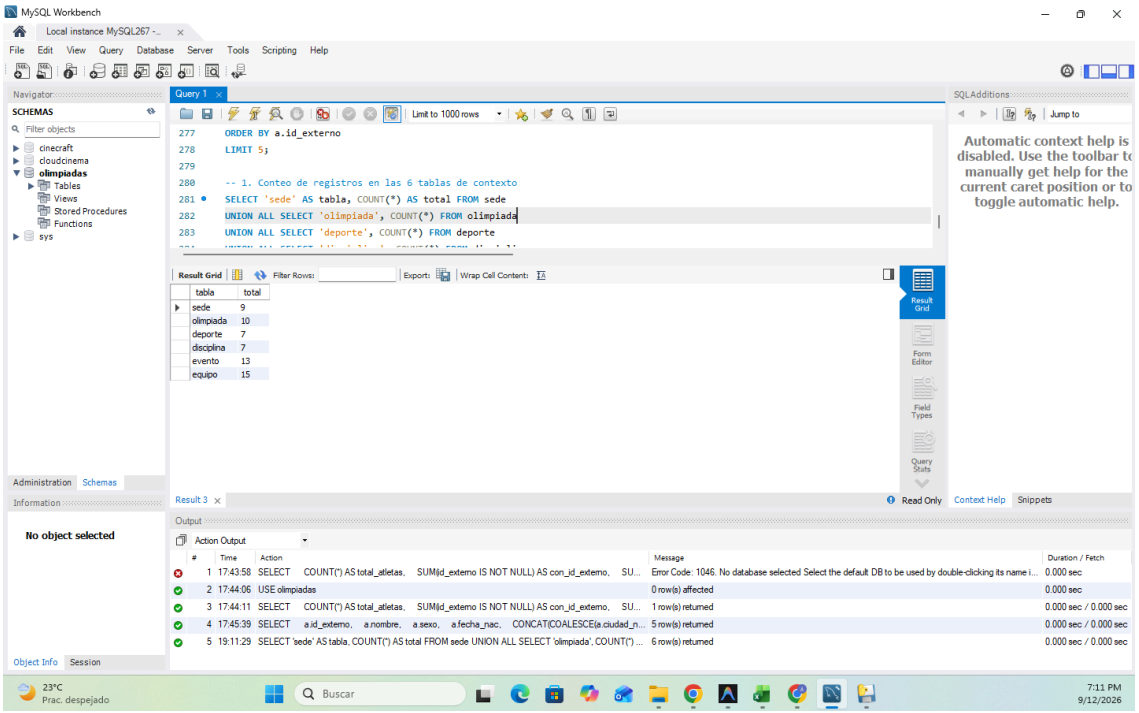
Se listan los paises cargados



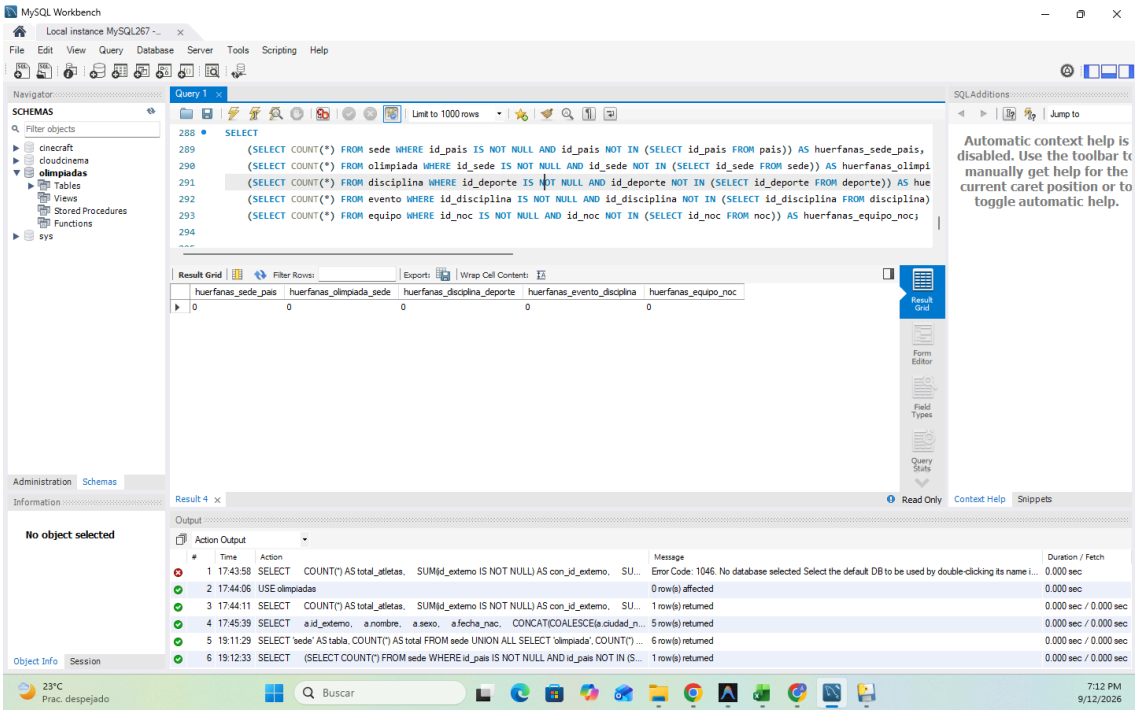
Script que demuestra que los comites olimpicos están correctamente relacionados con su pais



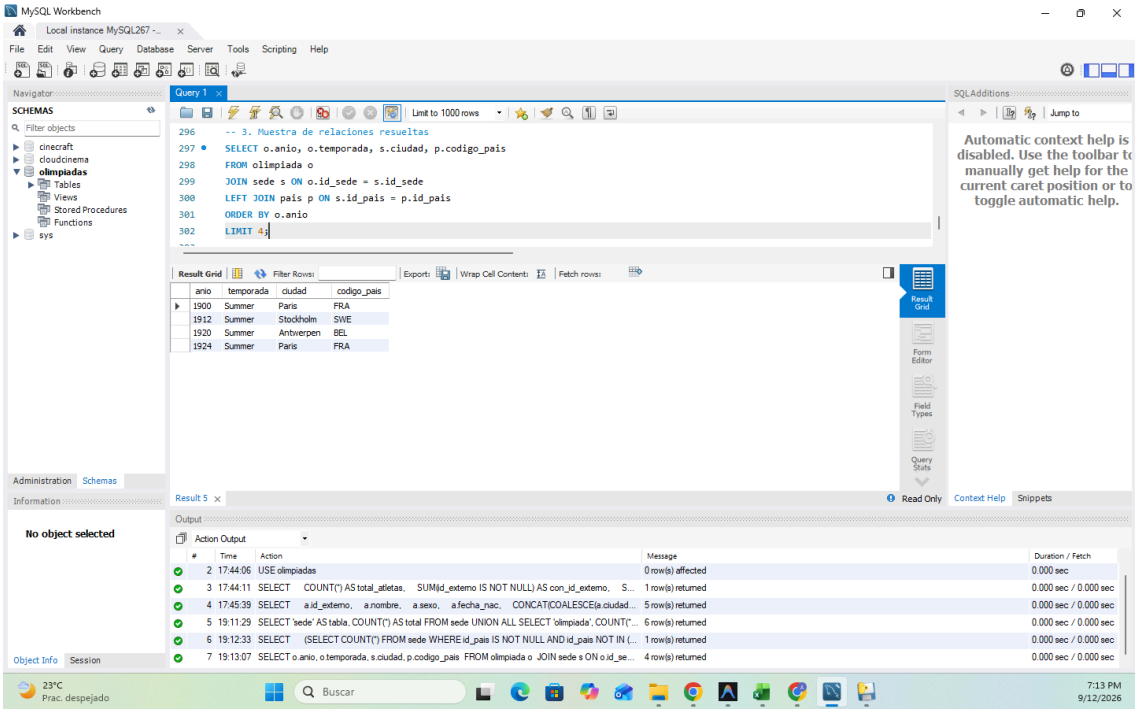
Se demuestra el correcto manejo del campo de años y conteo por pais



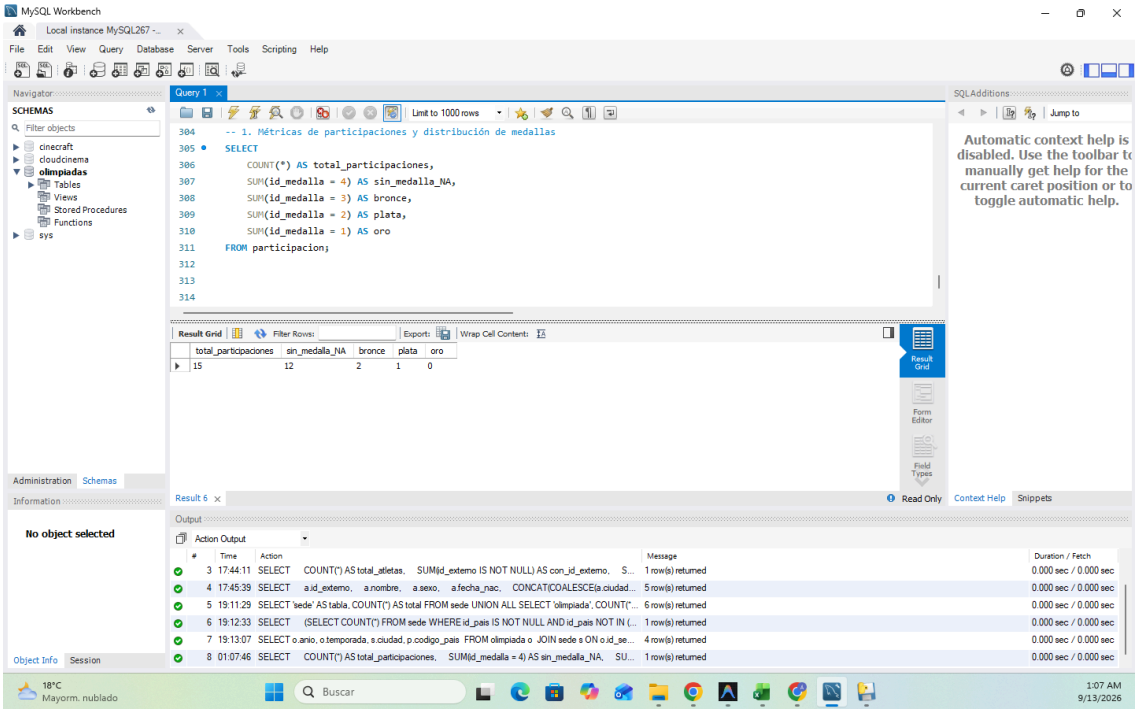
Conteo de registros en las 6 tablas de contexto, fase 3



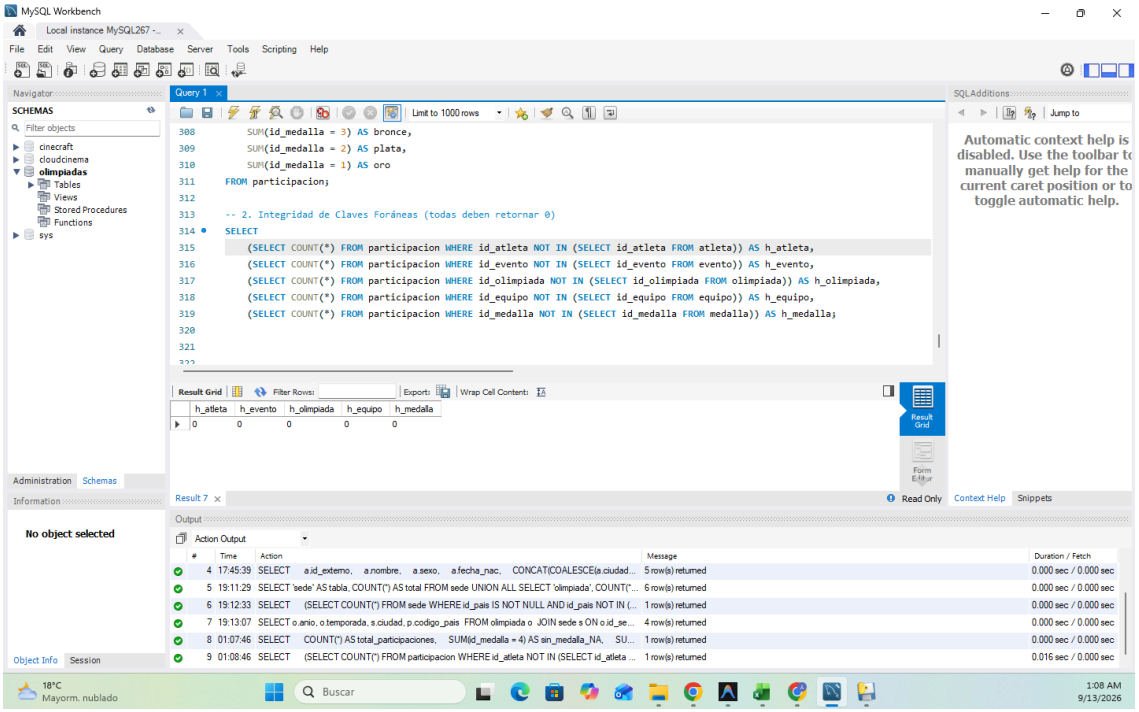
Integridad de Claves Foráneas (todas deben retornar 0 huérfanas)



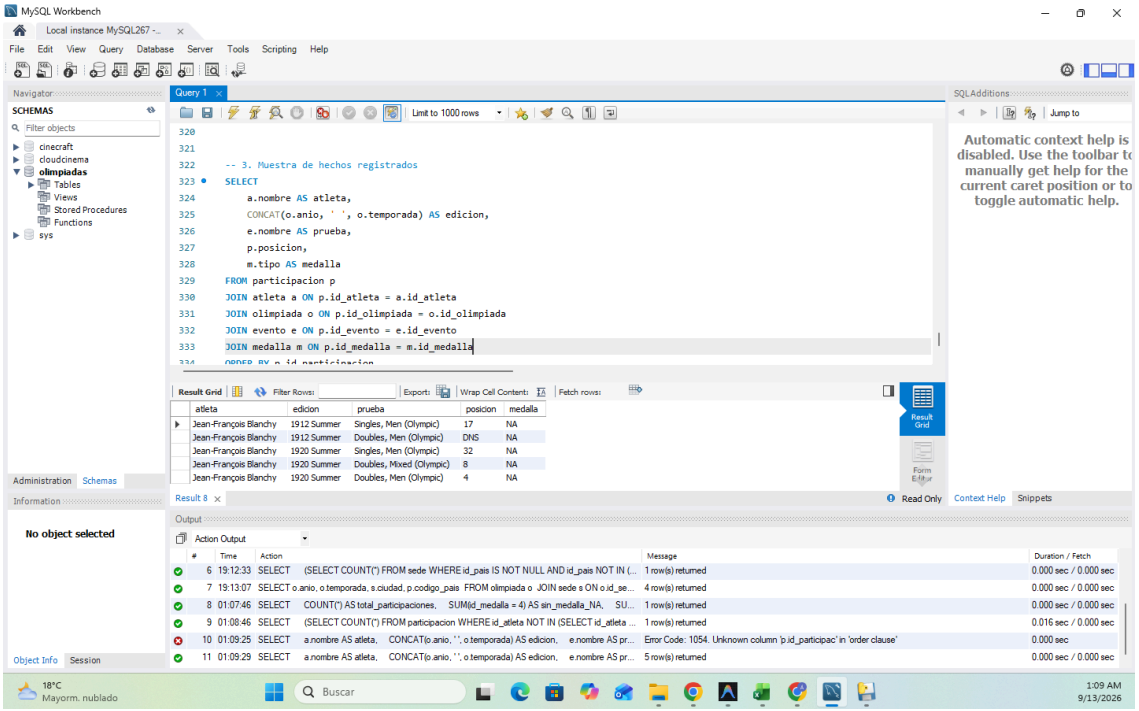
Muestra de relaciones resueltas



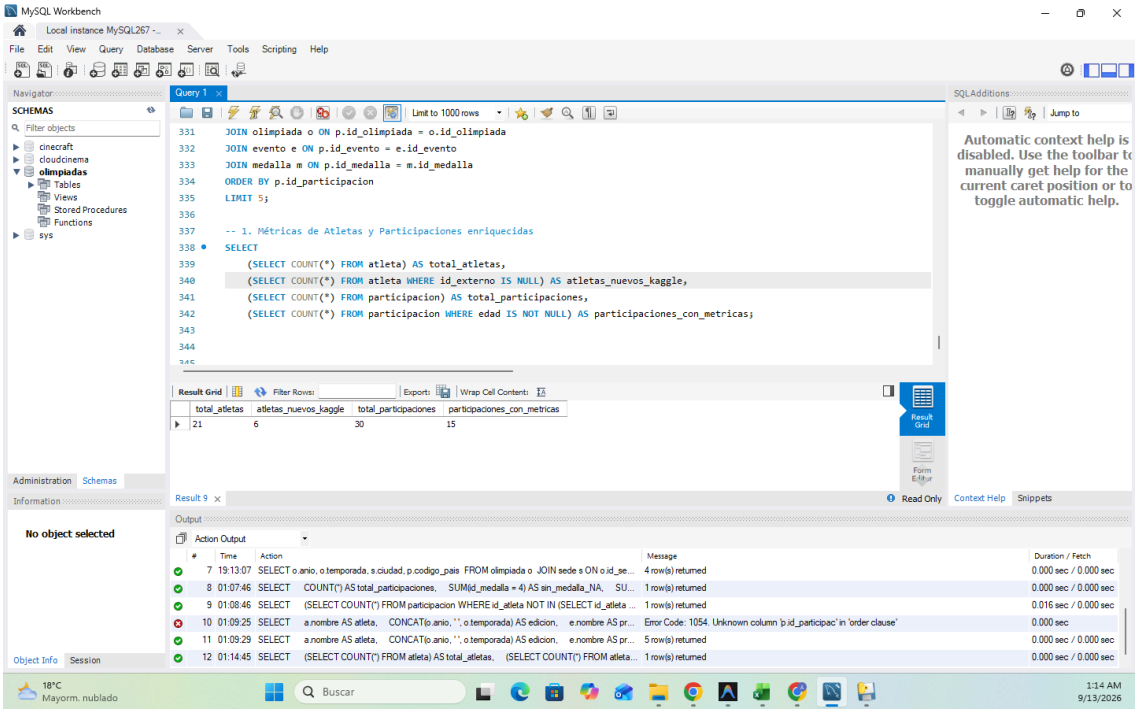
Métricas de participaciones y distribución de medallas



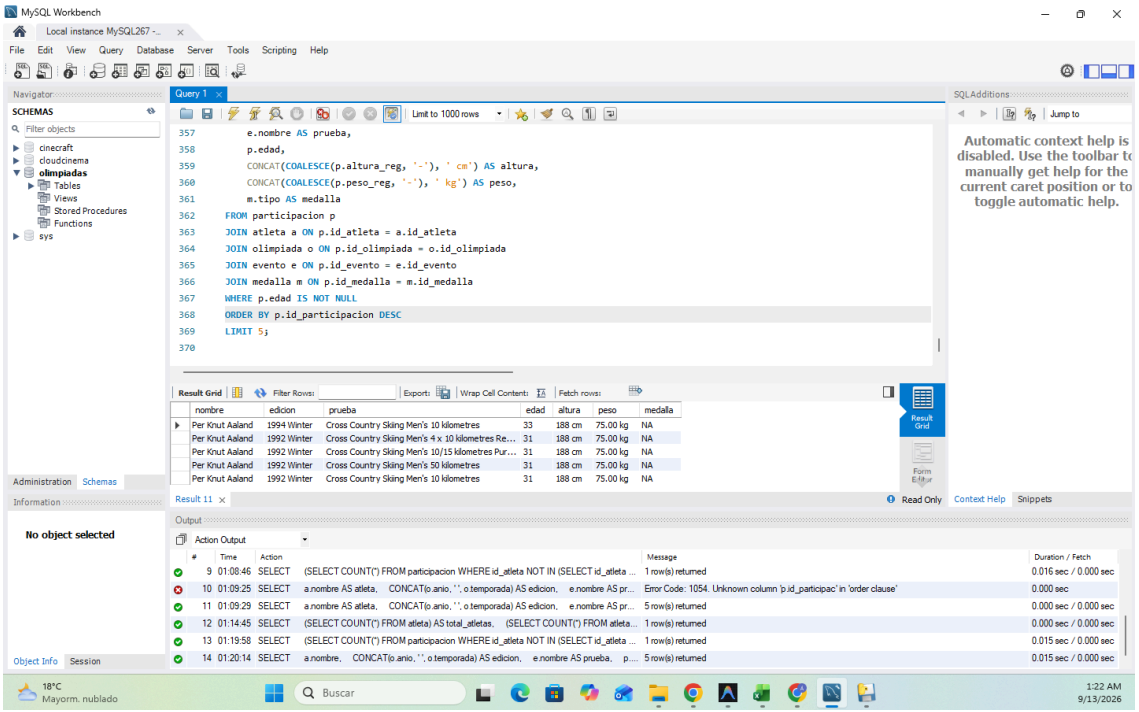
Integridad de Claves Foráneas (todas deben retornar 0)



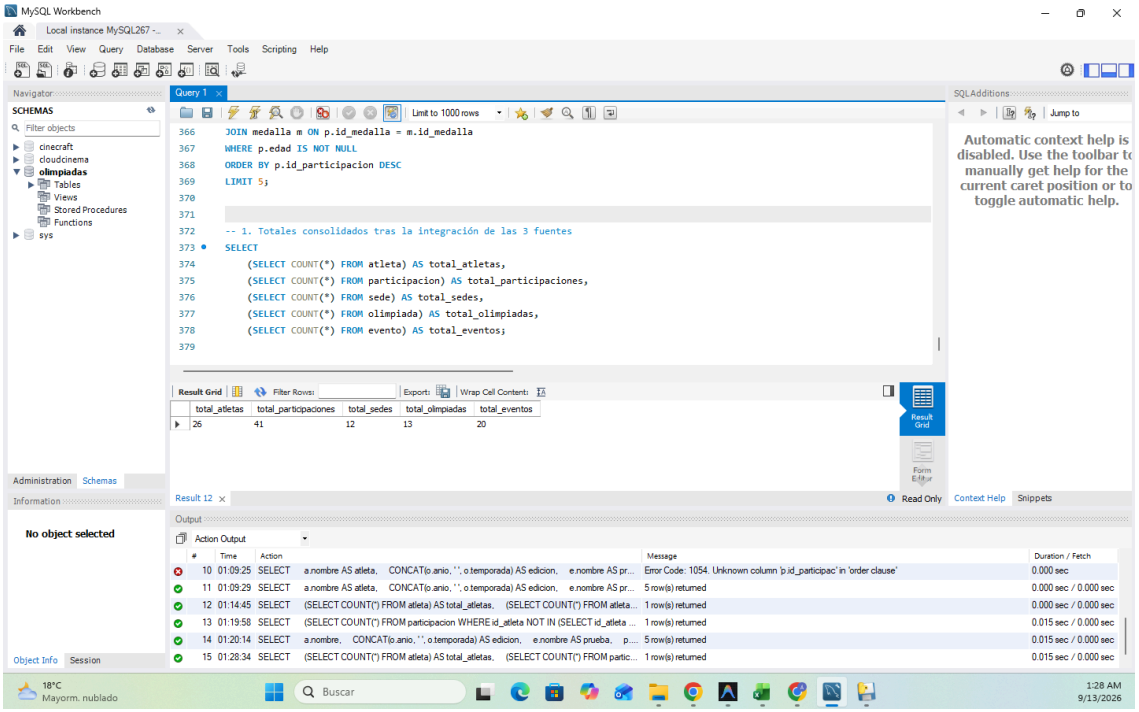
Muestra de hechos registrados



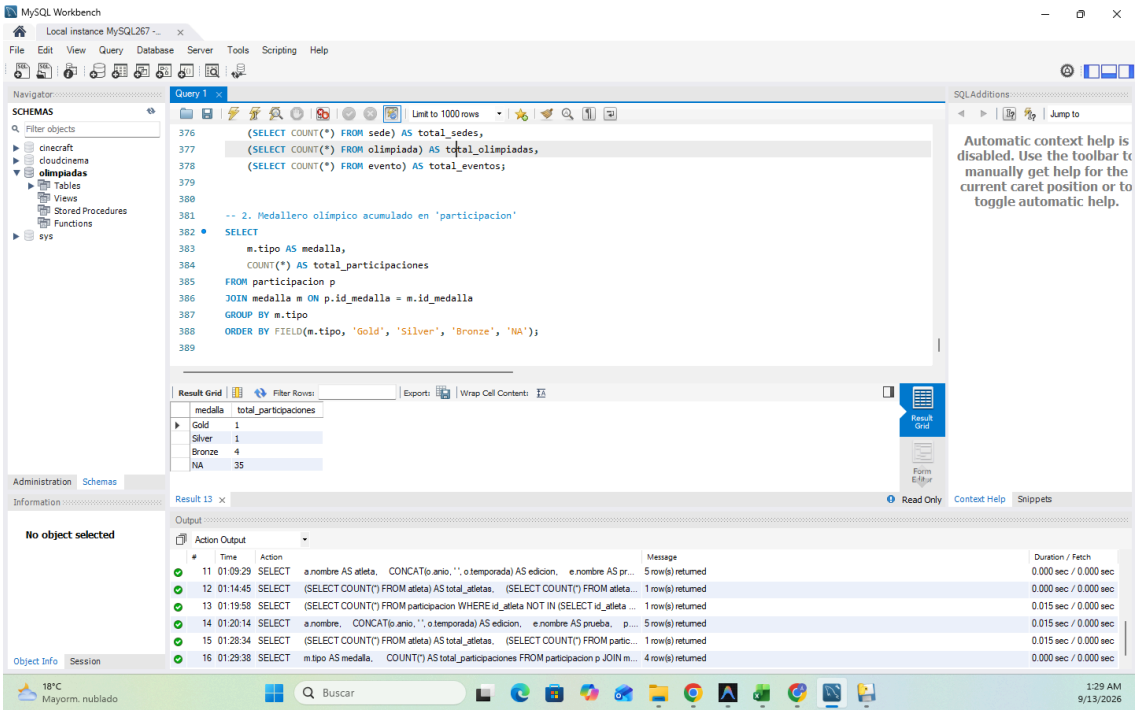
Métricas de Atletas y Participaciones enriquecidas



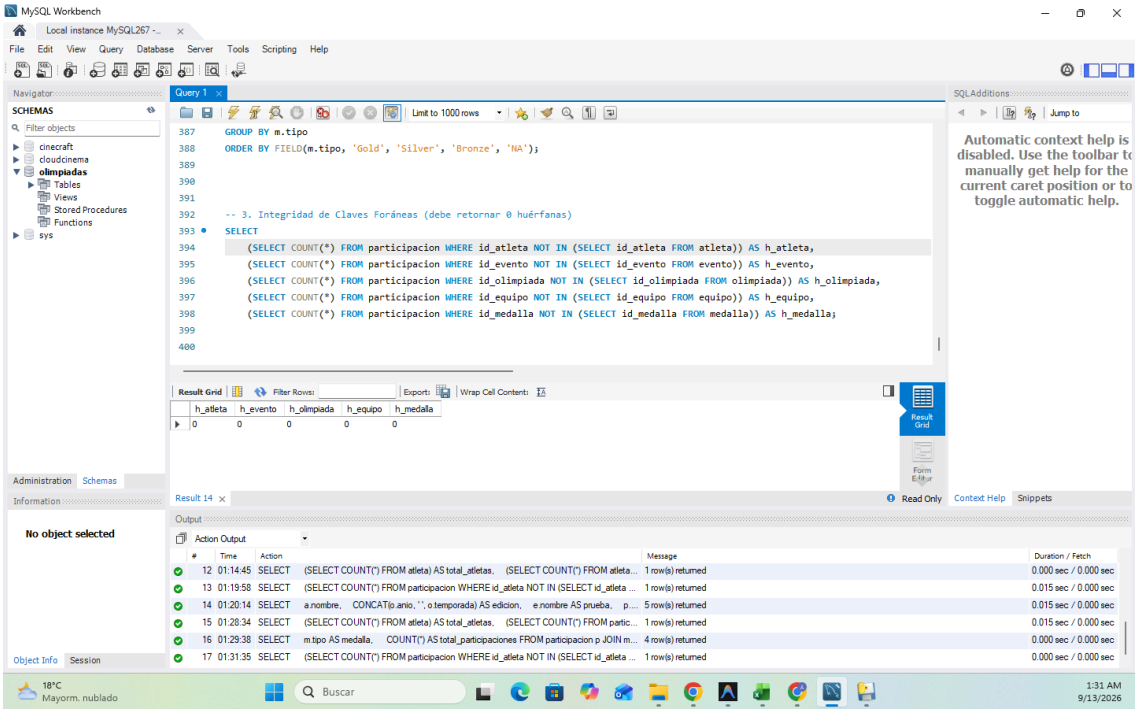
Muestra de participaciones enriquecidas con edad, altura y peso



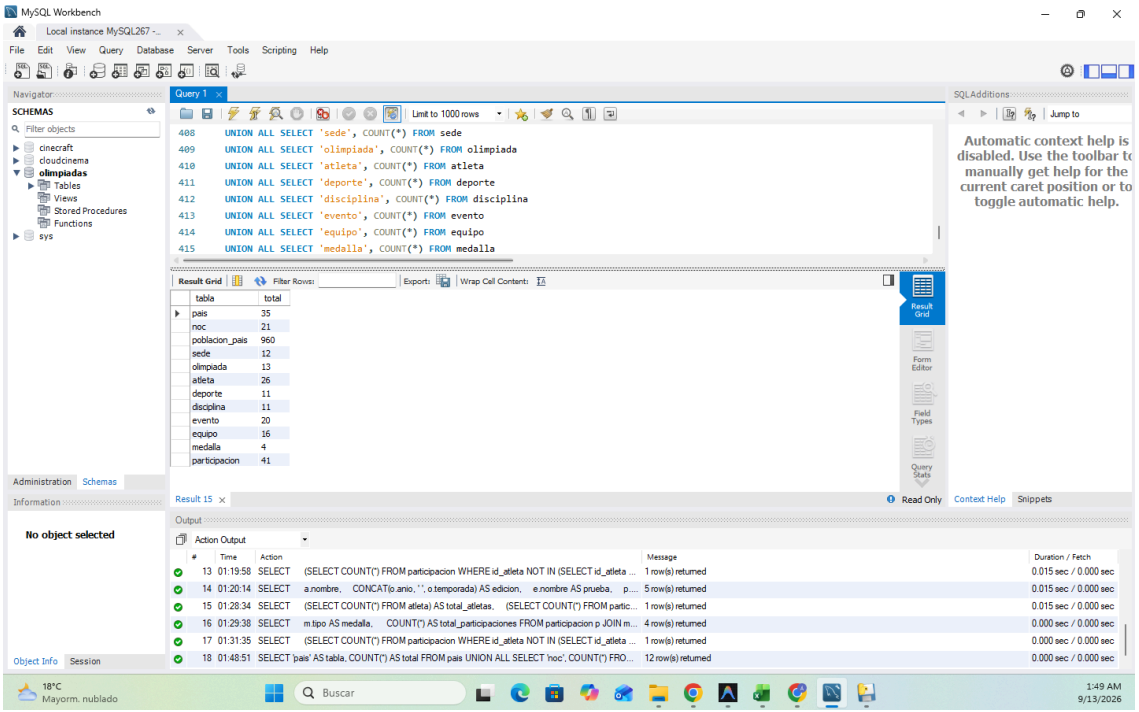
Totales consolidados tras la integración de las 3 fuentes



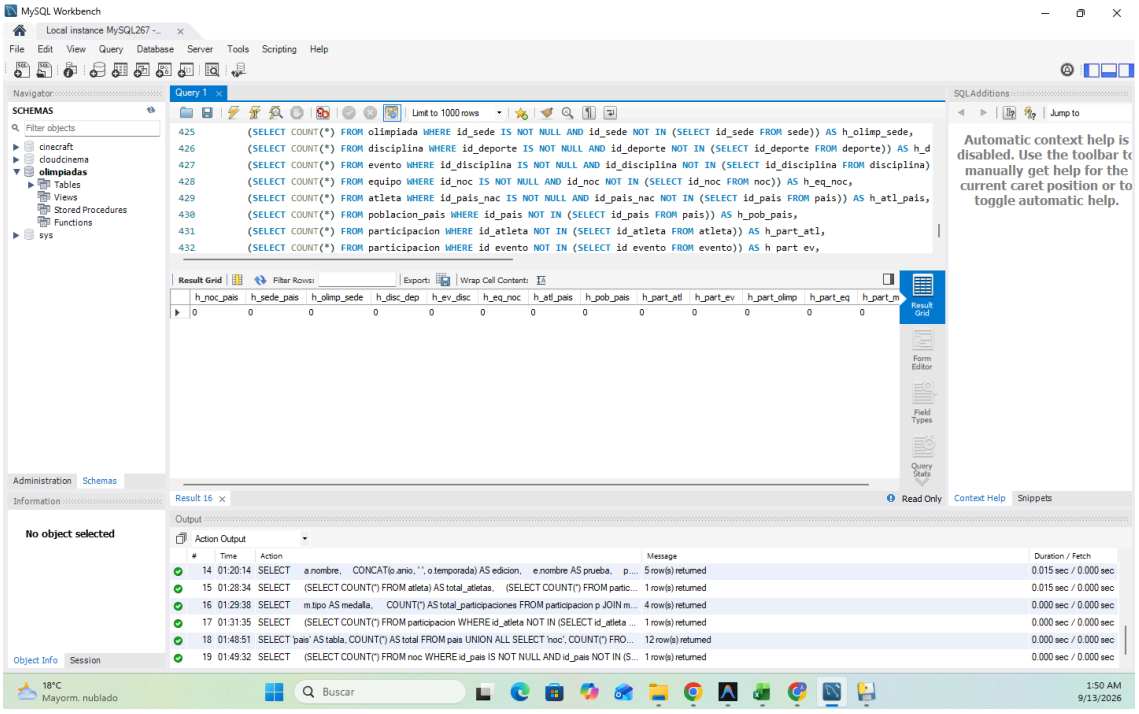
Medallero olímpico acumulado en 'participacion'



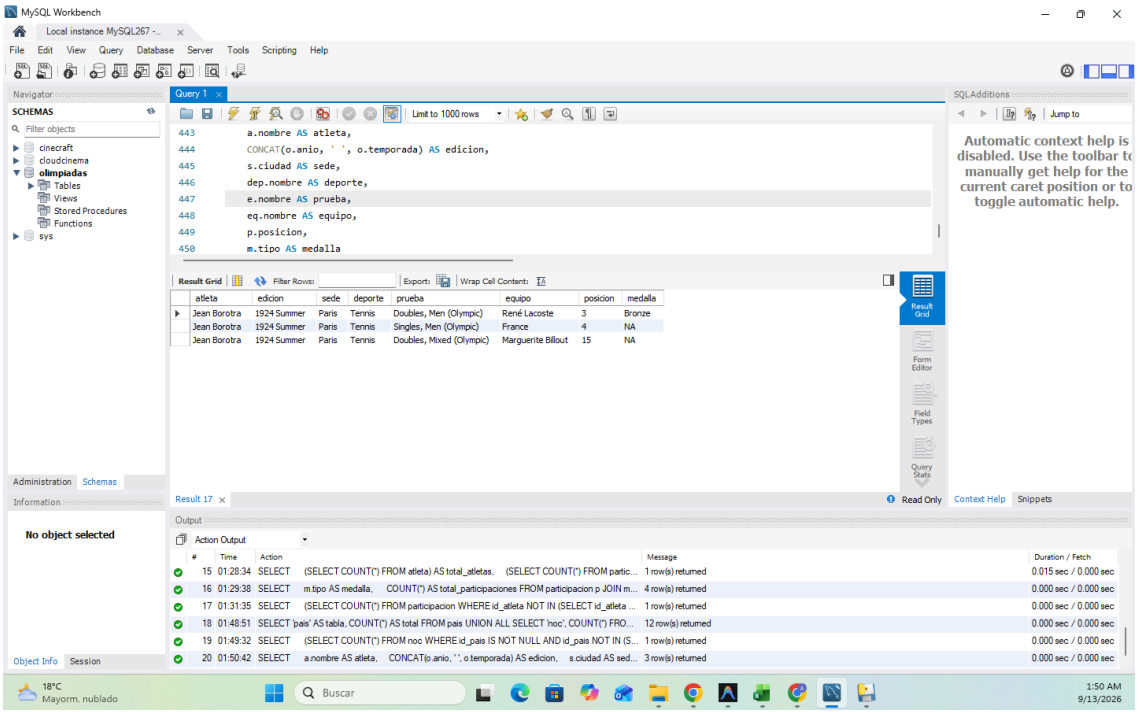
Integridad de Claves Foráneas (debe retornar 0 huérfanas)



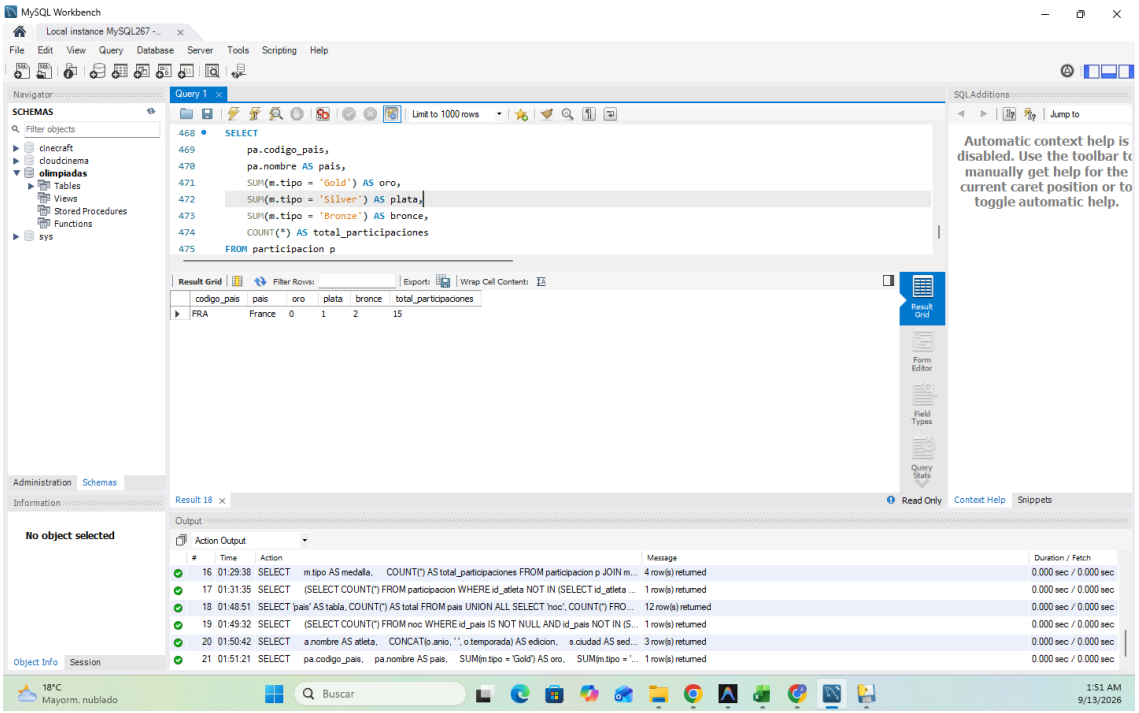
CENSO GLOBAL DE LAS 12 TABLAS POBLADAS



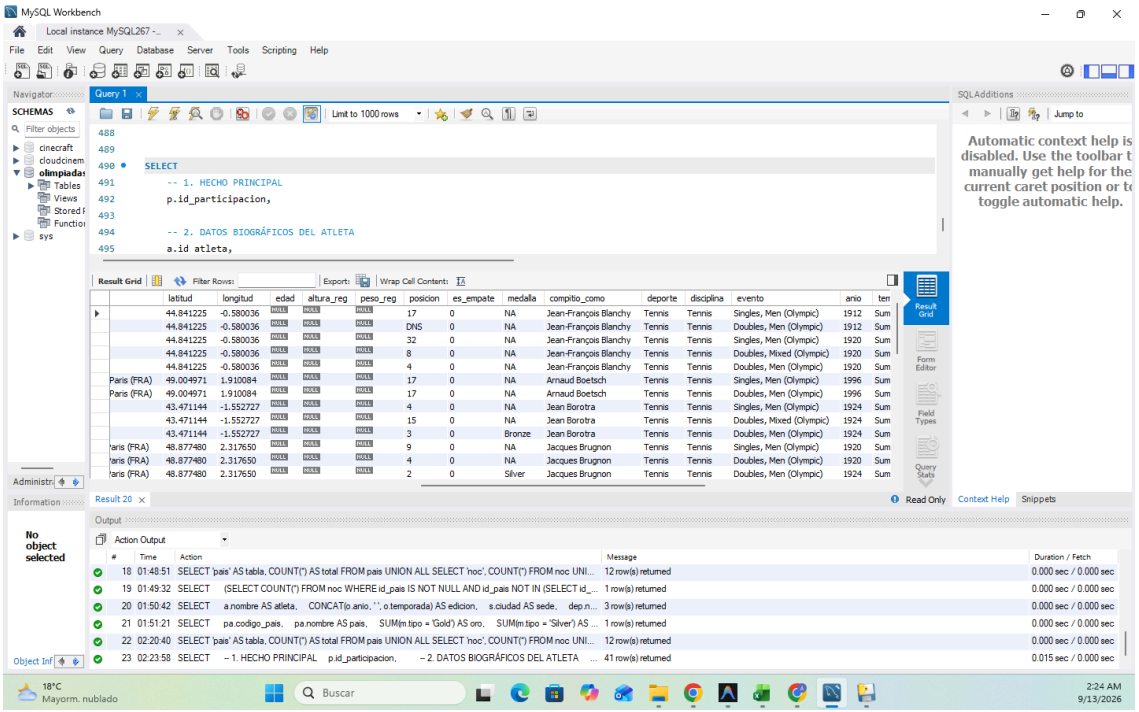
AUDITORÍA DE LAS 13 CLAVES FORÁNEAS (Todas deben dar 0 huérfanas)



SIMULACIÓN 'sp_consultar_atleta' (Búsqueda de 'Jean Borotra')

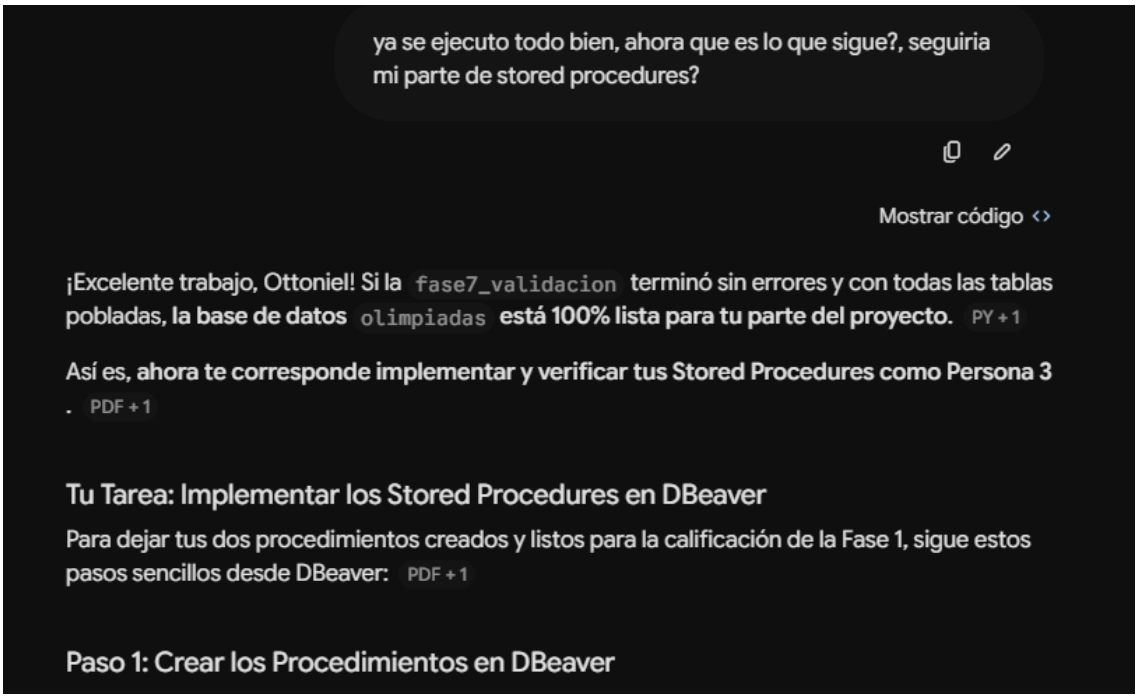


SIMULACIÓN 'sp_consultar_pais' (Medallero Histórico de 'FRA')

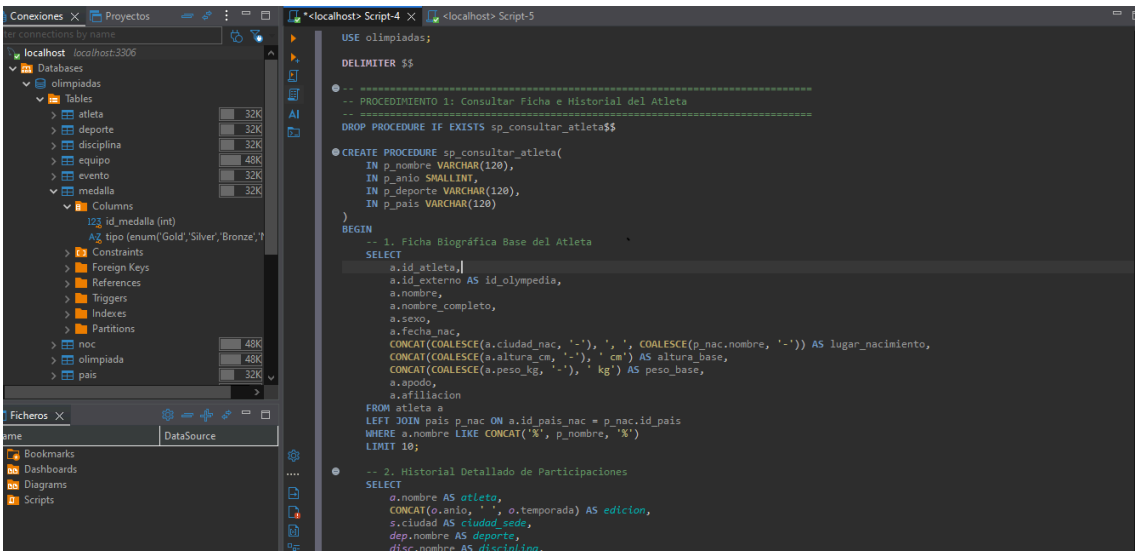


Vista completa de la base de datos

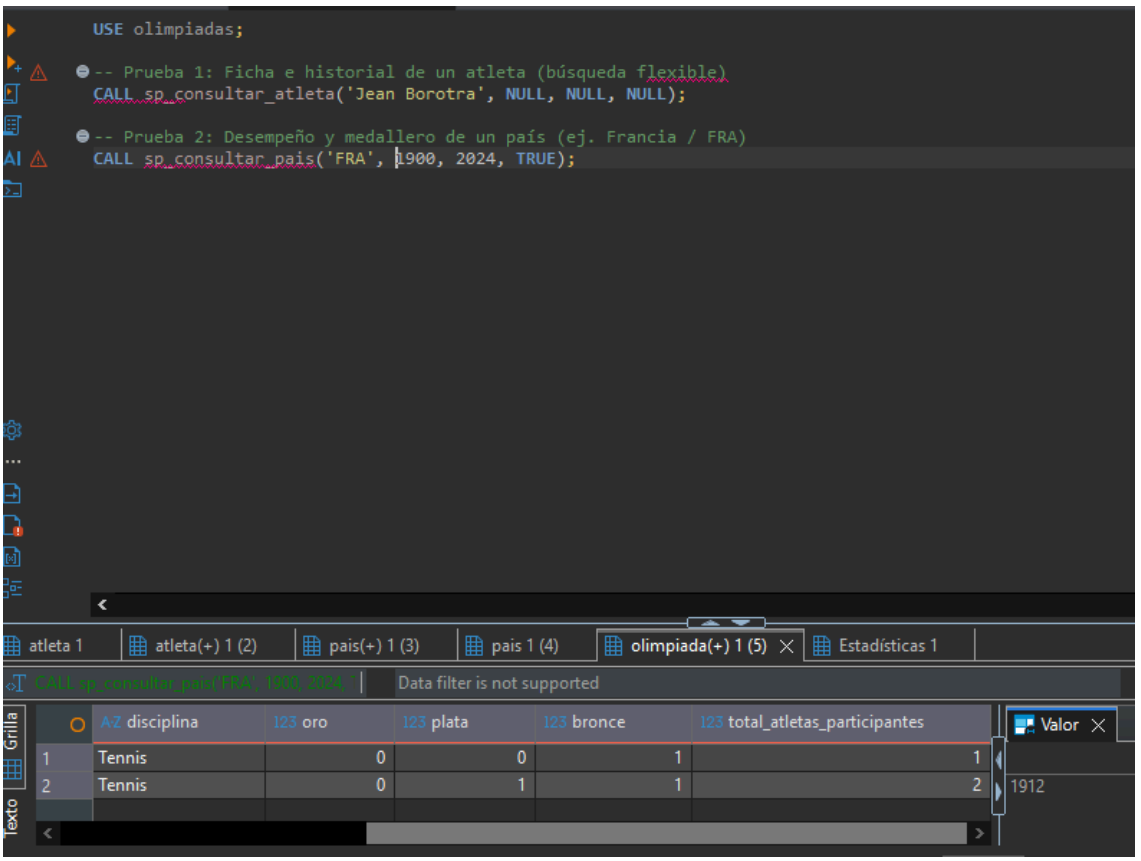
Para verificar que tenía ya todo completo levantado la base de datos con todos los requisitos que necesitaba, para que así pueda seguir con mi parte de Stored procedures.



Pongo el código que me dio y lo ejecuto



para verificar que en verdad se haya hecho bien, se comienza a hacer pruebas de verificación con algunas consultas



Ahora que ya verificamos que algunas consultas si hayan funcionado, pedimos que transforme todo el código en un script de python y así luego probarlo y ver si funciona bien con el script

y para ejecutarlo con python? no sería mejor eso, me refería un script para ejecutarlo con python

Mostrar código <>

¡Totalmente de acuerdo! Crear un script en Python para compilar y ejecutar tus Stored Procedures de forma automatizada es mucho más elegante, profesional y facilita enormemente la calificación de la **Persona 3.** PDF + 2

He creado el script `persona3_sp.py` integrado con la misma estructura y utilidades que usó tu grupo. PY + 2

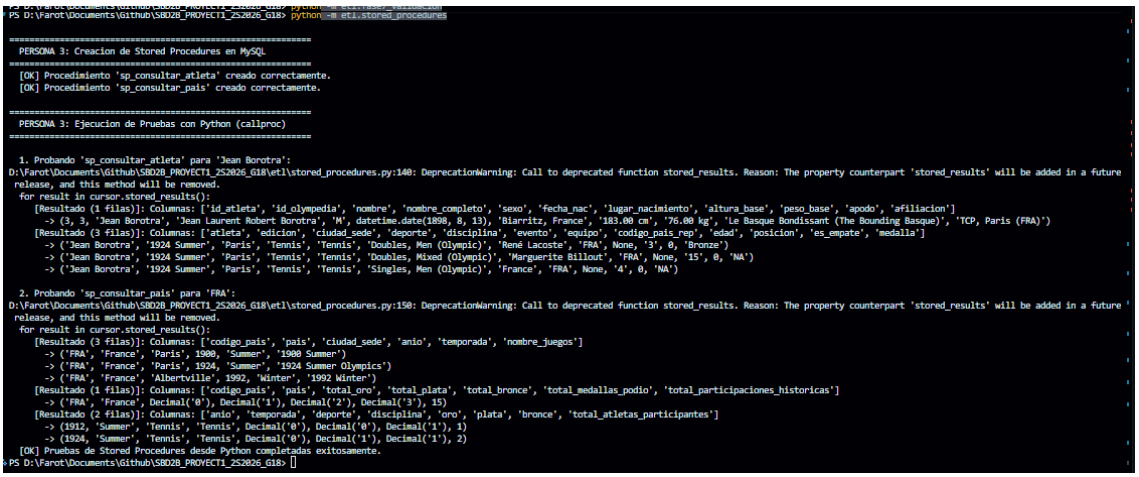
Código del Script: `persona3_sp.py`

Guarda este código en un archivo llamado `persona3_sp.py` en la carpeta de tu proyecto (o dentro del directorio `etl/`):

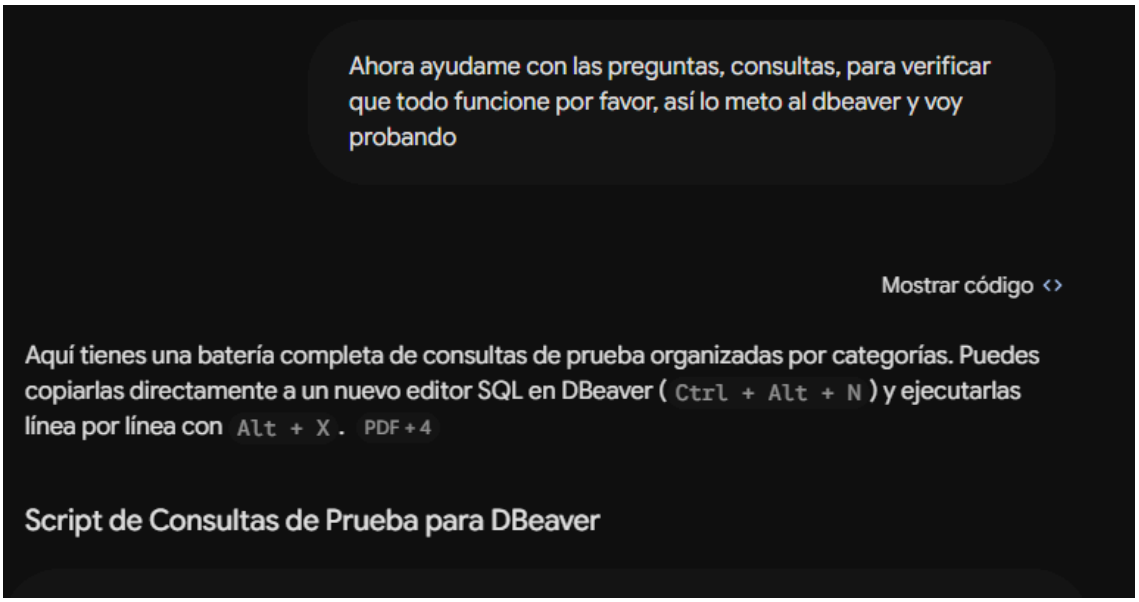
Código que dio del script



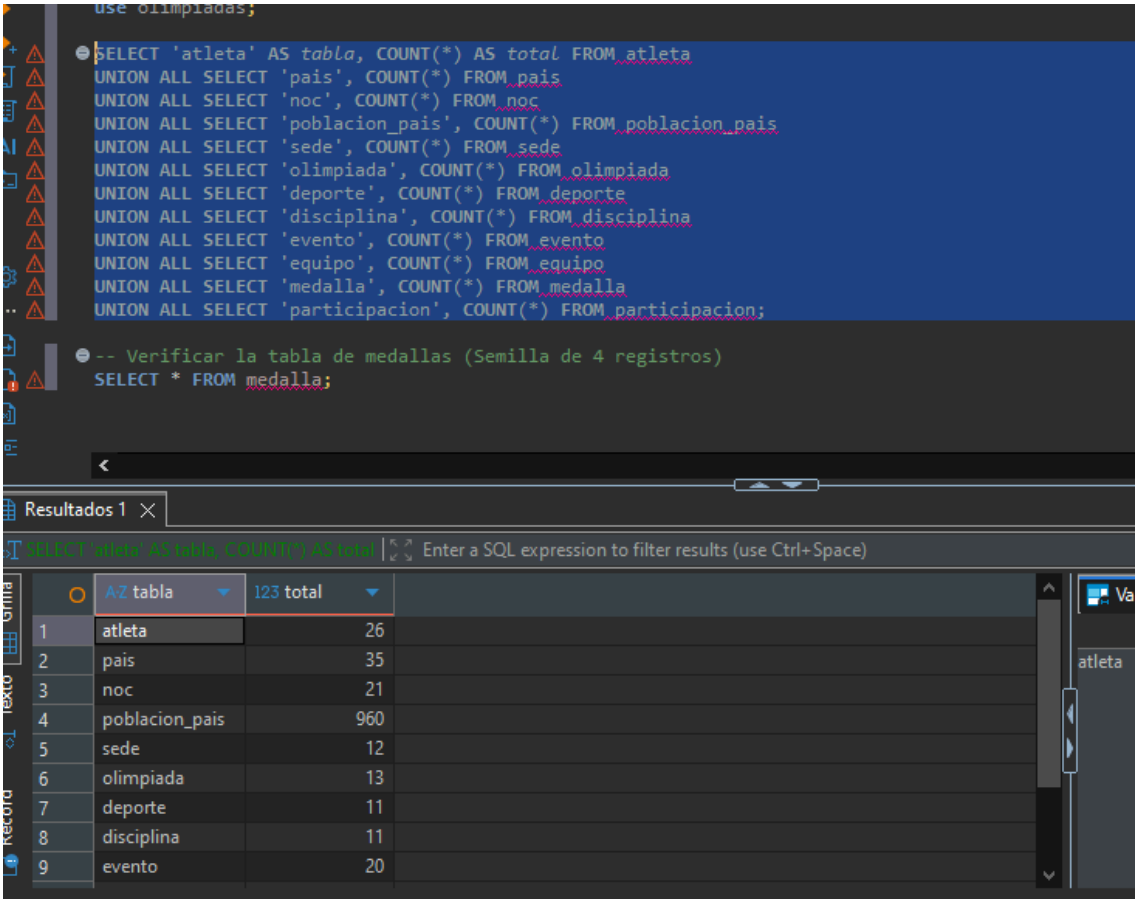
Ahora tocaría verificar si en verdad funciona con el script de python

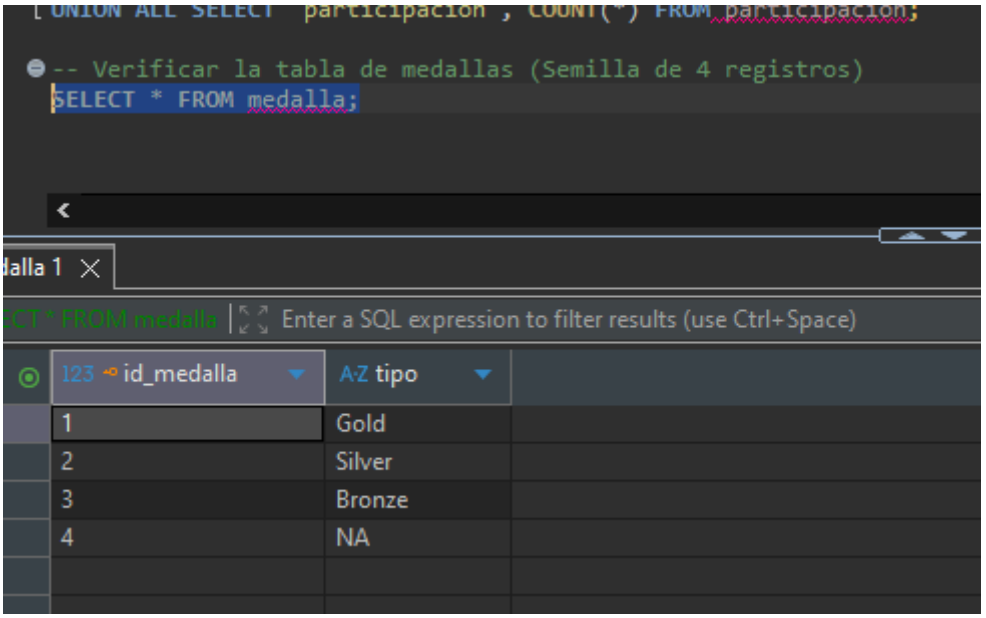


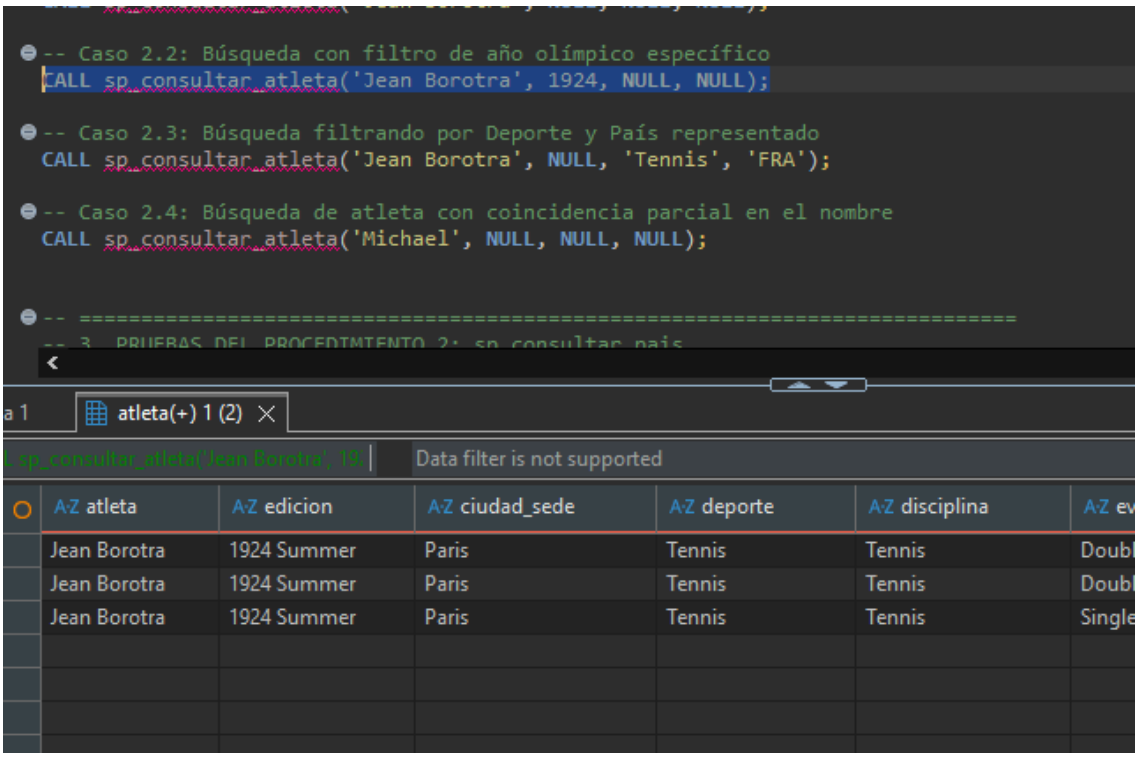
Ahora para verificar que en verdad de haya hecho todo bien, osea que la base de datos se haya creado bien y también con todos los requisitos indispensables



Se verifica lo que ne me dio la IA y también que todo este completo con la base de datos







```
-- Caso 2.2: Búsqueda con filtro de año olímpico específico
CALL sp_consultar_atleta('Jean Borotra', 1924, NULL, NULL);

-- Caso 2.3: Búsqueda filtrando por Deporte y País representado
CALL sp_consultar_atleta('Jean Borotra', NULL, 'Tennis', 'FRA');

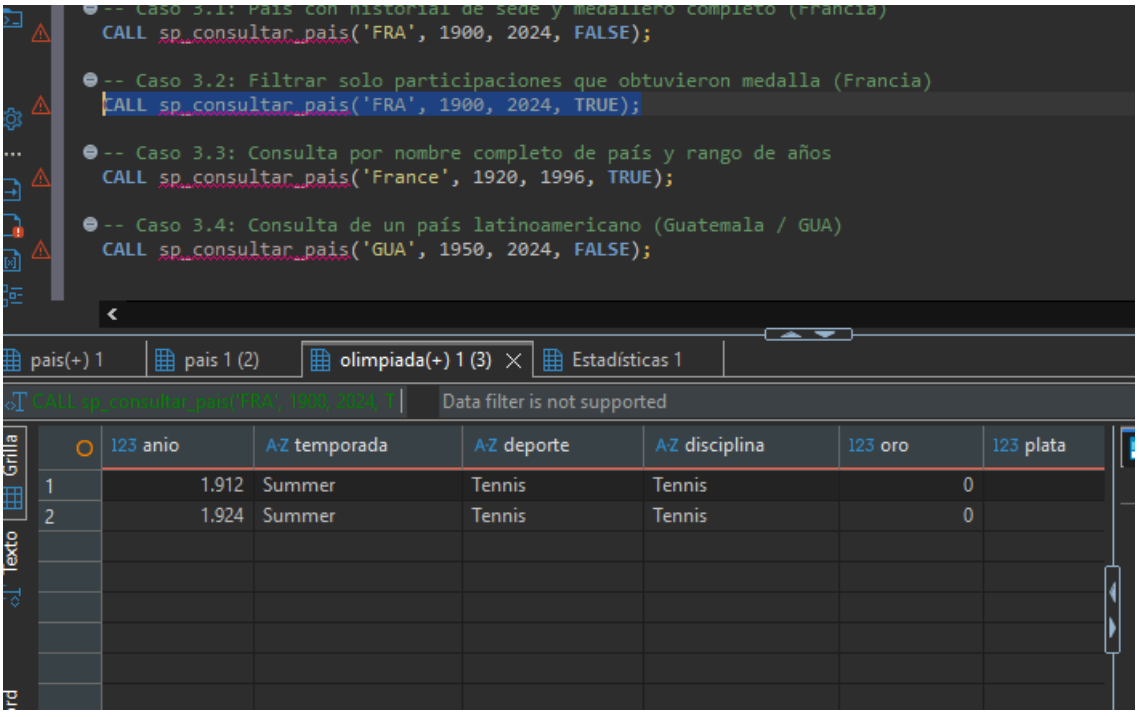
-- Caso 2.4: Búsqueda de atleta con coincidencia parcial en el nombre
CALL sp_consultar_atleta('Michael', NULL, NULL, NULL);

-- ========================================================================
-- 3. PRUEBAS DEL PROCEDIMIENTO 2: sp_consultar_pais
```

atleta(+) 1 (2)

Data filter is not supported

atleta	edicion	ciudad_sede	deporte	disciplina	ev
Jean Borotra	1924 Summer	Paris	Tennis	Tennis	Doubl
Jean Borotra	1924 Summer	Paris	Tennis	Tennis	Doubl
Jean Borotra	1924 Summer	Paris	Tennis	Tennis	Single



```
-- Caso 3.1: País con historial de sede y medallero completo (Francia)
CALL sp_consultar_pais('FRA', 1900, 2024, FALSE);

-- Caso 3.2: Filtrar solo participaciones que obtuvieron medalla (Francia)
CALL sp_consultar_pais('FRA', 1900, 2024, TRUE);

-- Caso 3.3: Consulta por nombre completo de país y rango de años
CALL sp_consultar_pais('France', 1920, 1996, TRUE);

-- Caso 3.4: Consulta de un país latinoamericano (Guatemala / GUA)
CALL sp_consultar_pais('GUA', 1950, 2024, FALSE);
```

pais(+) 1 | pais 1 (2) | olimpiada(+) 1 (3) | Estadísticas 1

CALL sp_consultar_pais('FRA', 1900, 2024, T

Data filter is not supported

	anio	temporada	deporte	disciplina	oro	plata
1	1.912	Summer	Tennis	Tennis	0	
2	1.924	Summer	Tennis	Tennis	0	

```sql
-- 4.1 Top 5 Atletas con mayor número de medallas obtenidas
SELECT
    a.nombre AS atleta,
    pa.nombre AS pais_nacimiento,
    SUM(m.tipo = 'Gold') AS oro,
    SUM(m.tipo = 'Silver') AS plata,
    SUM(m.tipo = 'Bronze') AS bronce,
    COUNT(*) AS total_medallas
FROM participacion p
JOIN atleta a ON p.id_atleta = a.id_atleta
LEFT JOIN pais pa ON a.id_pais_nac = pa.id_pais
JOIN medalla m ON p.id_medalla = m.id_medalla
WHERE m.tipo IN ('Gold', 'Silver', 'Bronze')
GROUP BY a.id_atleta, a.nombre, pa.nombre
ORDER BY total_medallas DESC, oro DESC
LIMIT 5;

-- 4.2 Ediciones olímpicas registradas con sus sedes y países anfitriones
SELECT
```

a(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

atleta	pais_nacimiento	oro	plata	bronce	tot
Arvo Aaltonen	[NULL]	0	0	2	
Edgar Lindenau Aabye	[NULL]	1	0	0	
Jacques Brugnon	France	0	1	0	
Jean Borotra	France	0	0	1	
Albert Canet	Great Britain	0	0	1	

Valo

Tex

Arvo

```sql
-- 4.2 Ediciones olímpicas registradas con sus sedes y países anfitriones
SELECT
    o.anio,
    o.temporada,
    o.nombre_juegos,
    s.ciudad AS ciudad_sede,
    p.nombre AS pais_sede
FROM olimpiada o
JOIN sede s ON o.id_sede = s.id_sede
LEFT JOIN pais p ON s.id_pais = p.id_pais
ORDER BY o.anio ASC;
```

piada(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

anio	temporada	nombre_juegos	ciudad_sede	pais_sede
1.900	Summer	1900 Summer	Paris	France
1.912	Summer	1912 Summer Olympics	Stockholm	Sweden
1.920	Summer	1920 Summer	Antwerpen	Belgium
1.924	Summer	1924 Summer Olympics	Paris	France
1.932	Summer	1932 Summer	Los Angeles	United States
1.952	Summer	1952 Summer	Helsinki	Finland
1.988	Winter	1988 Winter	Calgary	Canada
1.992	Winter	1992 Winter	Albertville	France

Valor

1900

novar | Save | Cancel | Exportar datos ... | 200